

ZNS: Avoiding the Block Interface Tax for Flash-based SSDs

Matias Bjørling, Abutalib Aghayev, Hans Holmberg, Aravind Ramesh,
Damien Le Moal, Gregory R. Ganger, George Amvrosiadis

2026. 08. 19

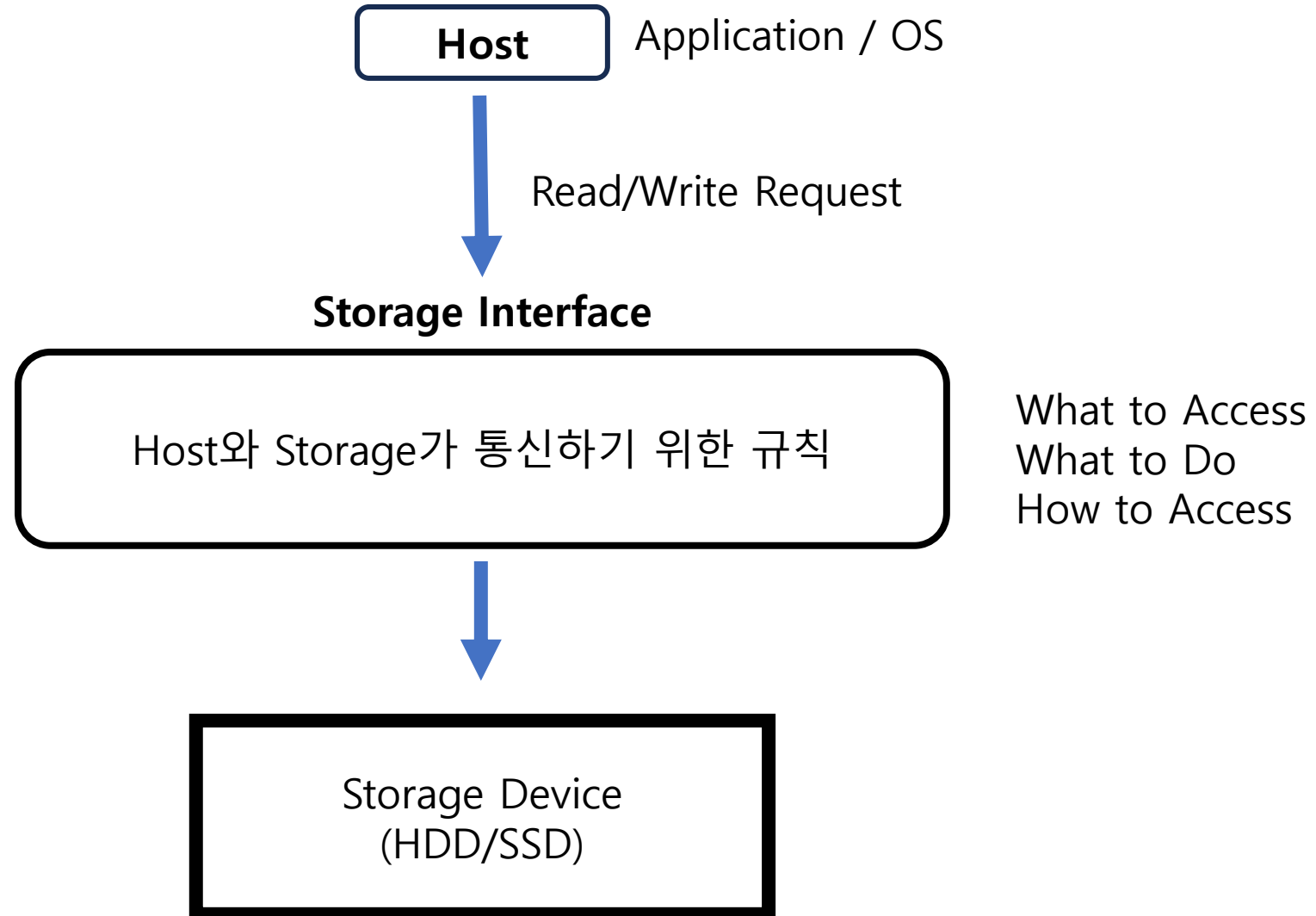
Presentation by Jung In, Minsu Kim

wjddls0318@dankook.ac.kr, minsk0515@dankook.ac.kr

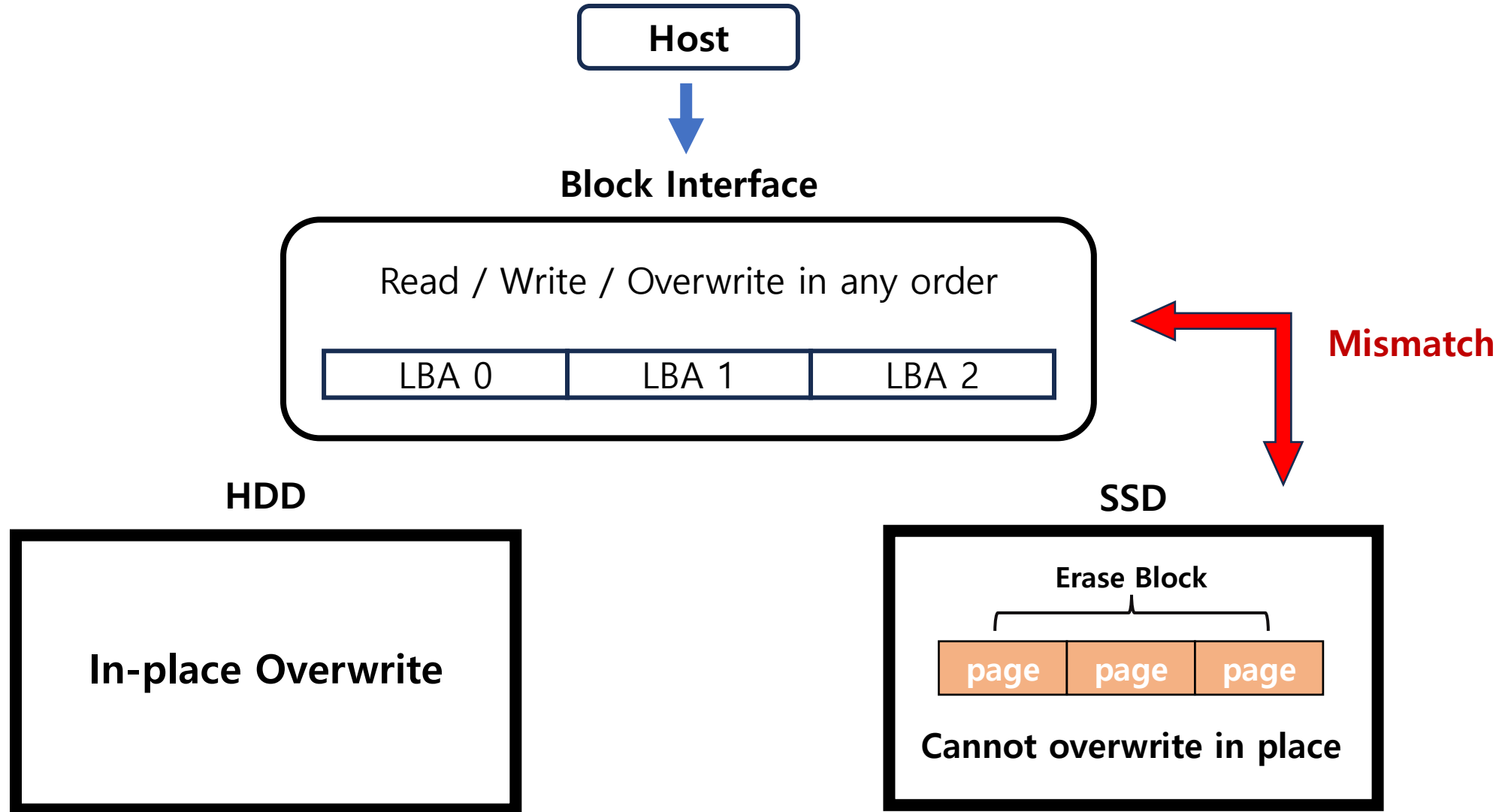
Contents

1. Problem Background
2. Evolving Towards ZNS
3. ZenFS
4. Conclusion

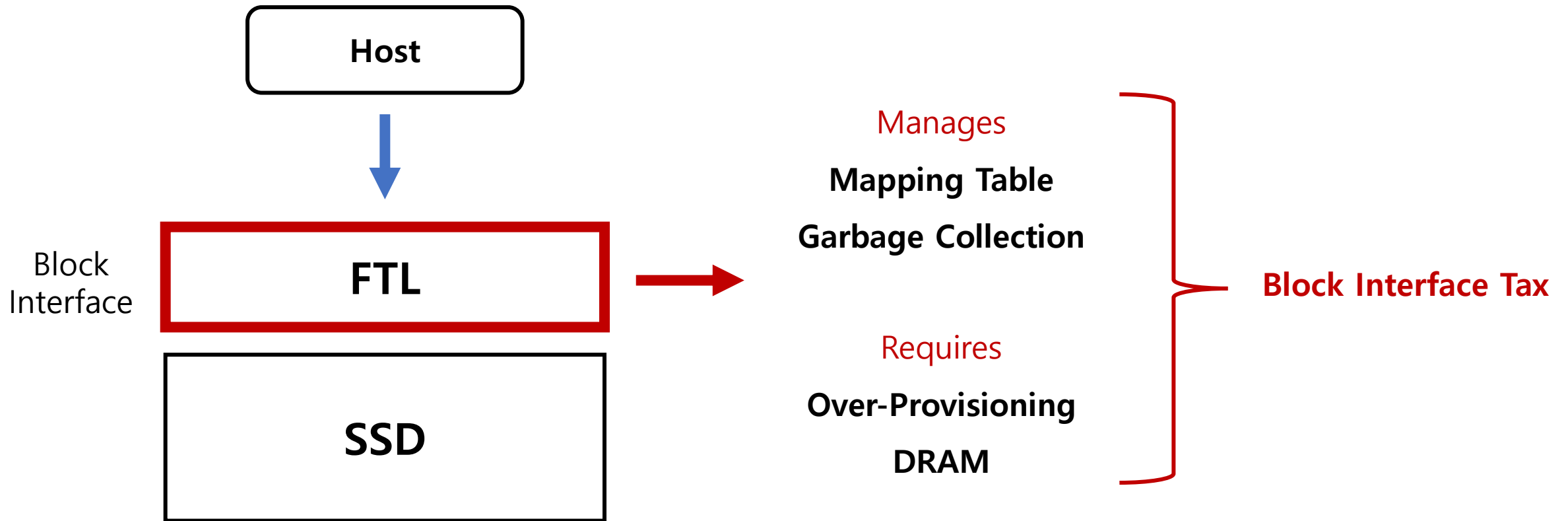
Problem Background - Storage Interface



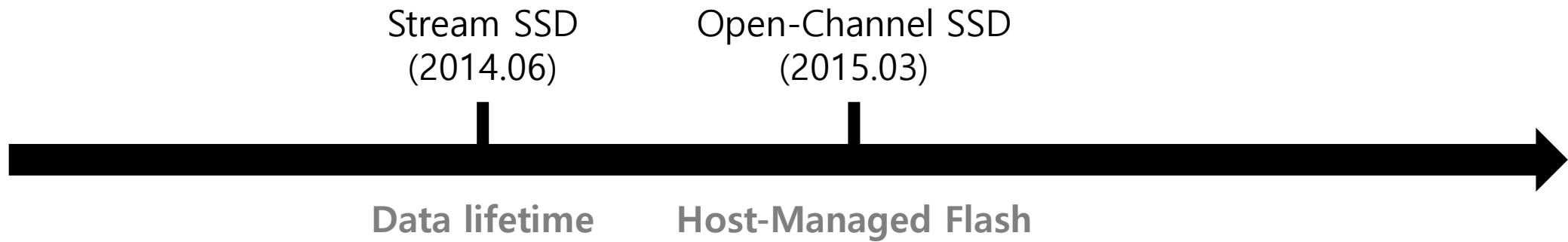
Problem Background - Block Interface Tax



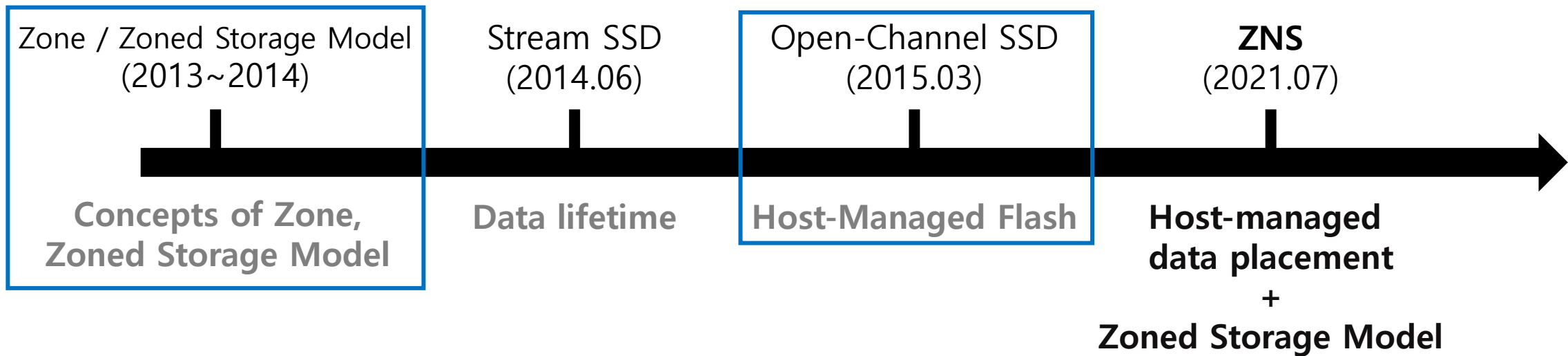
Problem Background - Block Interface Tax



Problem Background – Zone

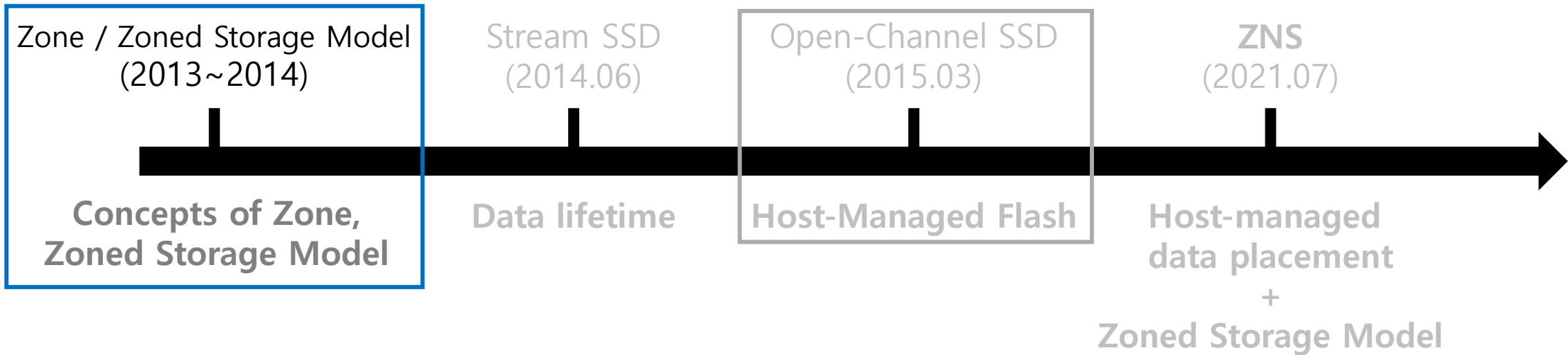


Problem Background – Zone



Standard Interface for NVMe SSDs

Problem Background – Zone



- Track 중첩 구조의 SMR HDD → 임의적인 Overwrite 제약
- Block Interface 대신 Zone Storage Model (+ SMR 계열 Interface)

Standard Interface for NVMe SSDs

Problem Background – Zone

여러 Logical Block을 포함하는 논리 주소 공간의 한 영역

Zone

Block

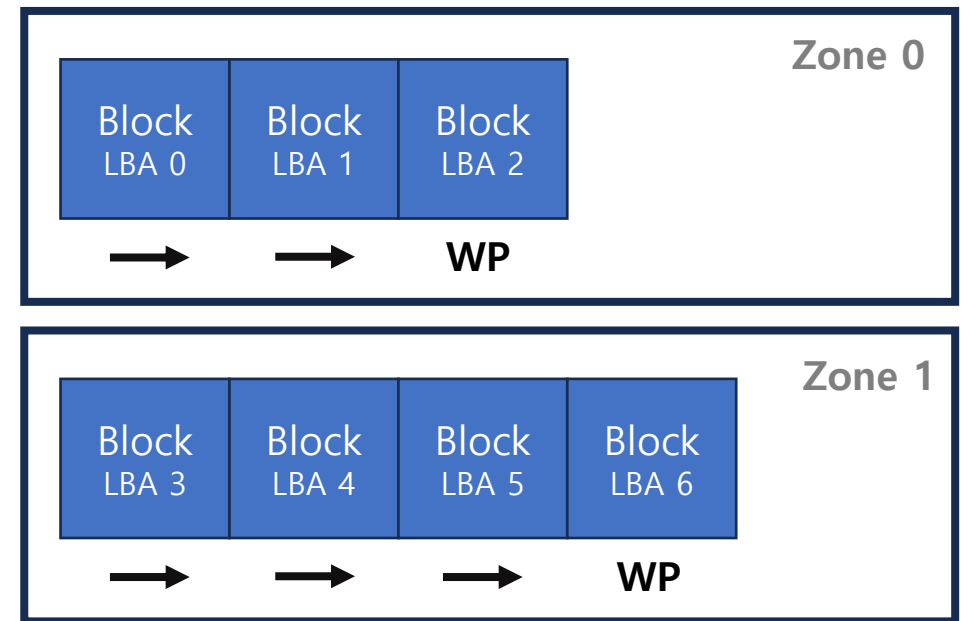


↑
write

↑
overwrite

↑
write

개별 Logical Block을 LBA로 자유롭게 접근



Logical Blocks를 Zone으로 그룹화
Zone별로 순차적으로 접근

Problem Background – Zone

Zone

- 정의
 - SSD의 논리 주소 공간의 일정 영역
- 핵심 제약 조건
 - **Sequential Write**만 가능
 - **Overwrite**는 Zone을 명시적으로 **Reset** 후에 가능
- 4가지 Zone 상태

Open Zone Limit

장치 자원/미디어 제약으로
Open 상태의 Zone 수를 제한

Empty

Open

Full

Problem Background – Zone

Zone

- 정의
 - SSD의 논리 주소 공간의 일정 영역
- 핵심 제약 조건
 - **Sequential Write**만 가능
 - **Overwrite**는 Zone을 명시적으로 **Reset** 후에 가능
- 4가지 Zone 상태

Open Zone Limit + Host의 New Zone Write



다른 Open Zone → **Closed** (자원 해제)



새로운 Zone Write (자원 할당)

Closed Zone은 Open 전환 시에 Write 가능



Problem Background – Zone

Zone Management

- Zone State Machine

- 4가지 Zone 상태 관리



- Write Pointer

- Zone 내부의 순차 쓰기 위치를 관리

- <핵심 동작>

- ✓ Write 성공 시 Write Pointer 갱신
- ✓ Write Pointer 외 위치에 Write 또는 FULL Zone에 Write는 실패 → 순차 Write 보장

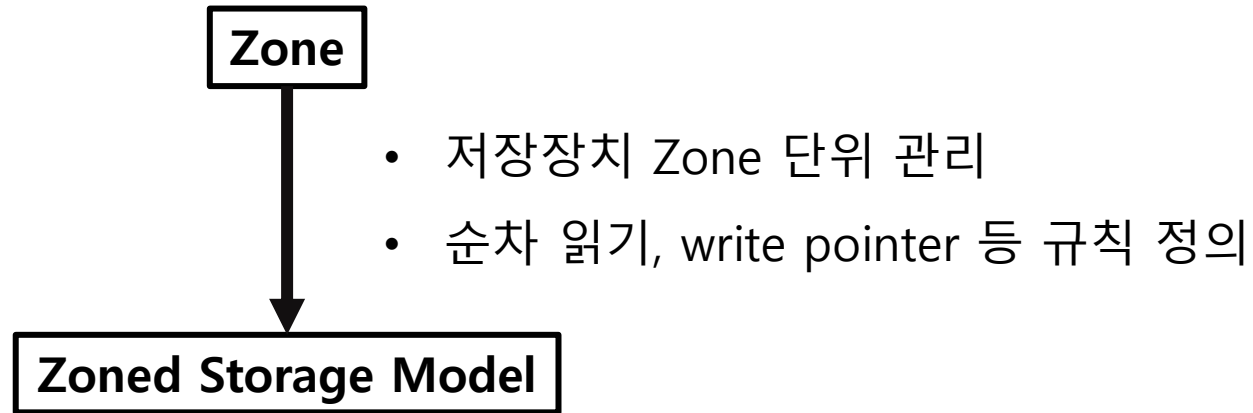
Write Pointer

- 종합 기여

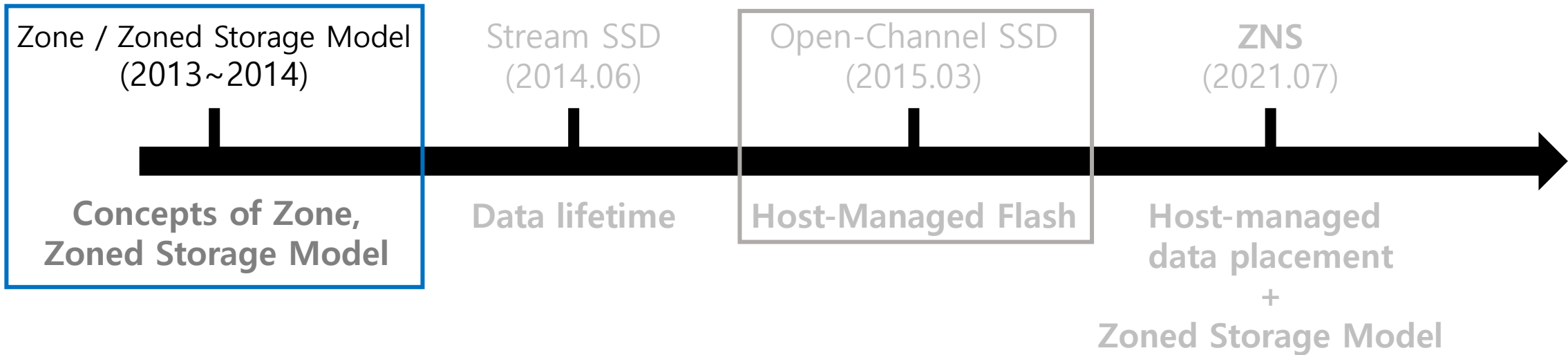
- Host가 Zone의 마지막 LBA 기록 위치를 추적 → Page 단위 Mapping 부담 감소

Problem Background – Zone

Zoned Storage Model



Problem Background – Zone



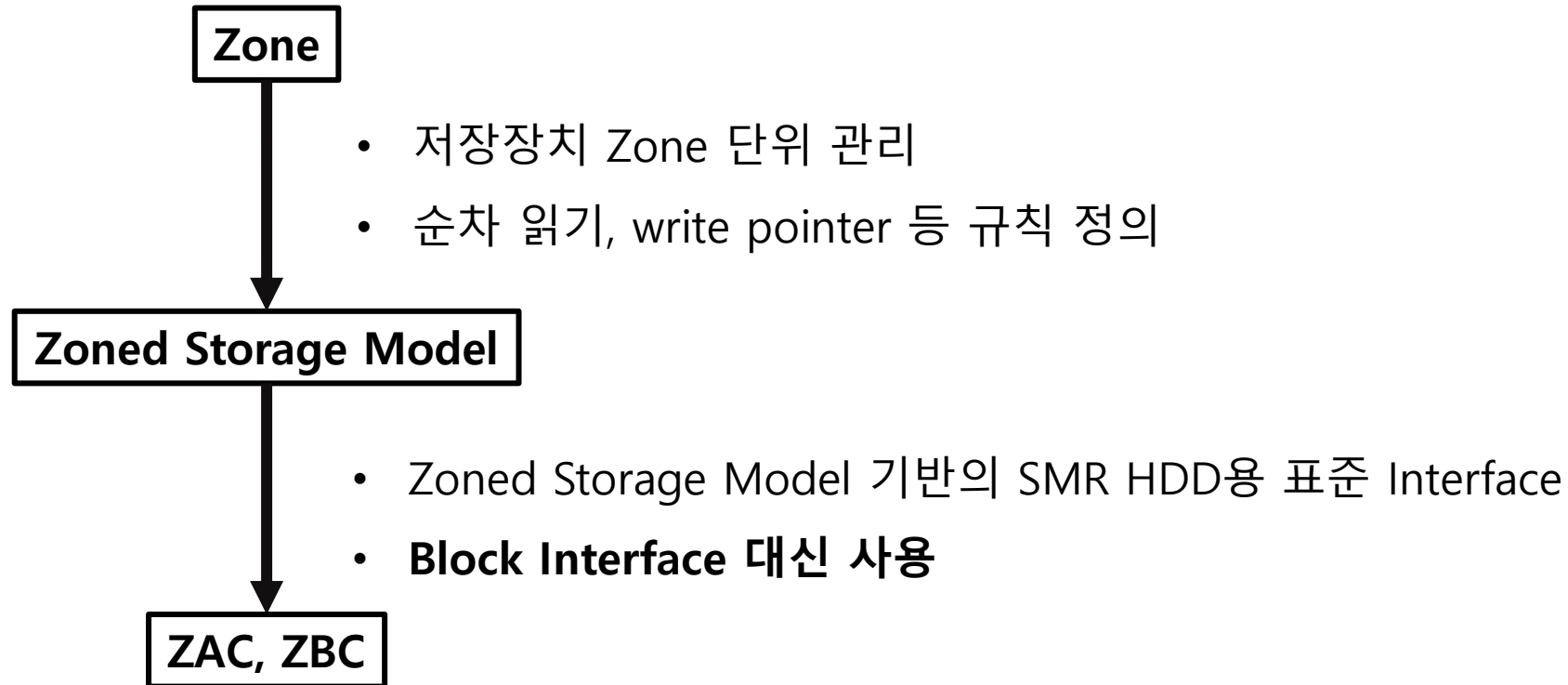
- Track 중첩 구조의 SMR HDD → 임의적인 Overwrite 제약

- Block Interface 대신 Zone Storage Model (+ SMR 계열 Interface)

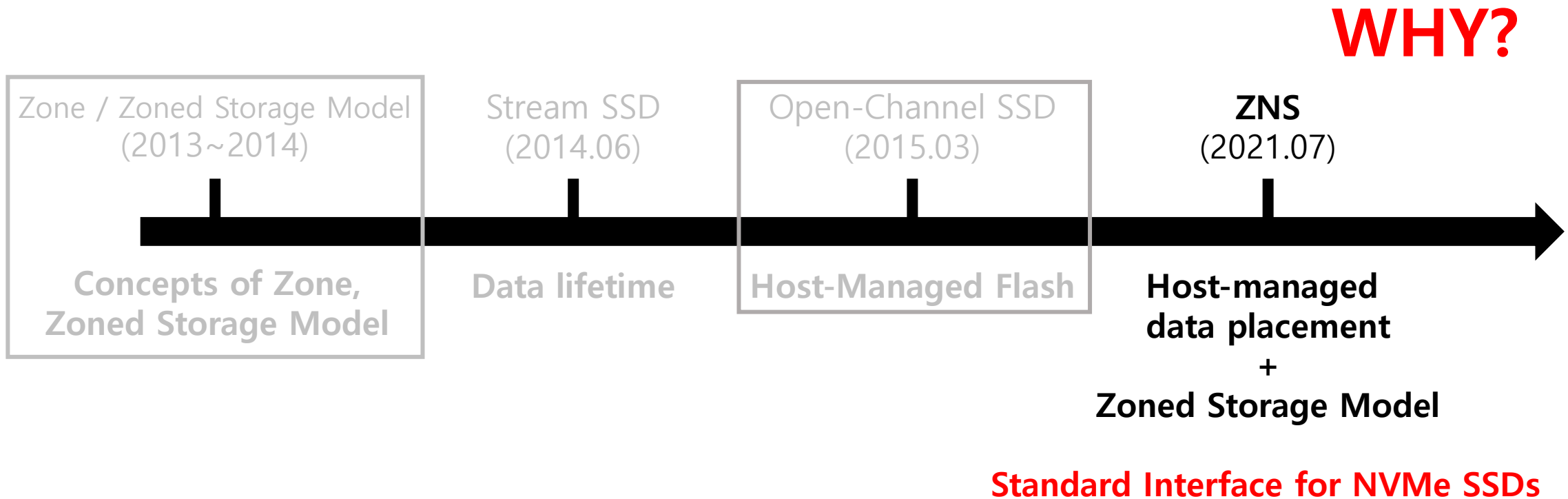
Standard Interface for NVMe SSDs

Problem Background – Zone

Zoned Storage Model

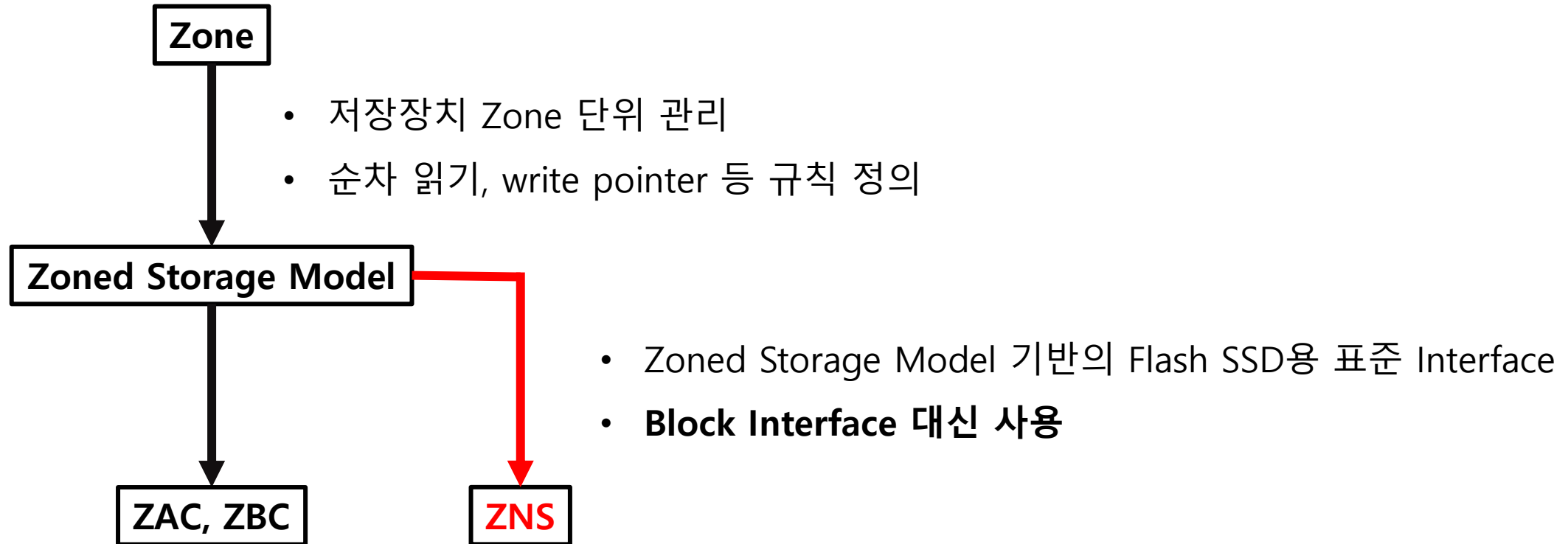


Problem Background – ZNS



Problem Background – ZNS

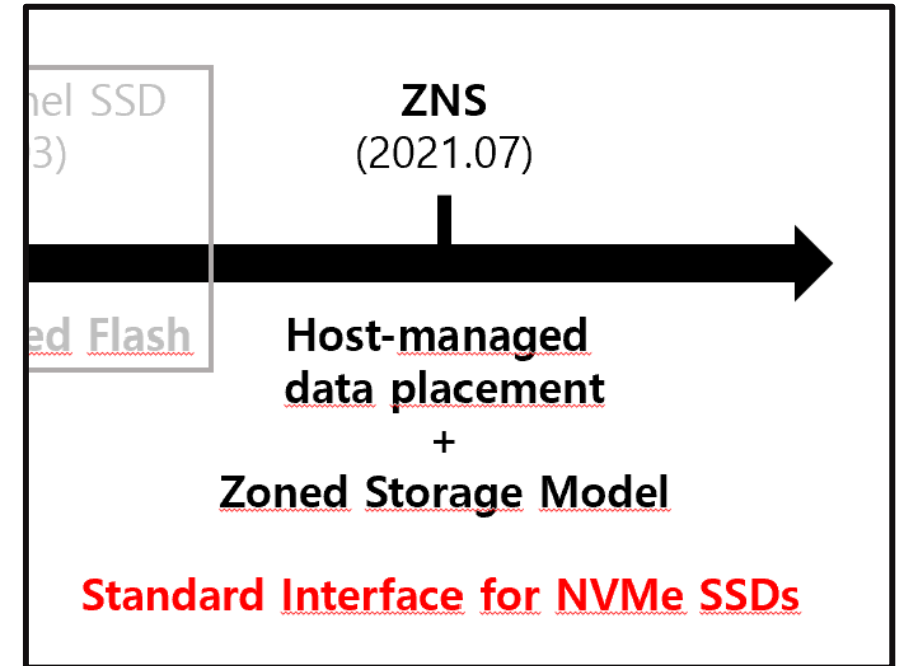
ZNS (Zoned Namespace)



Problem Background – ZNS

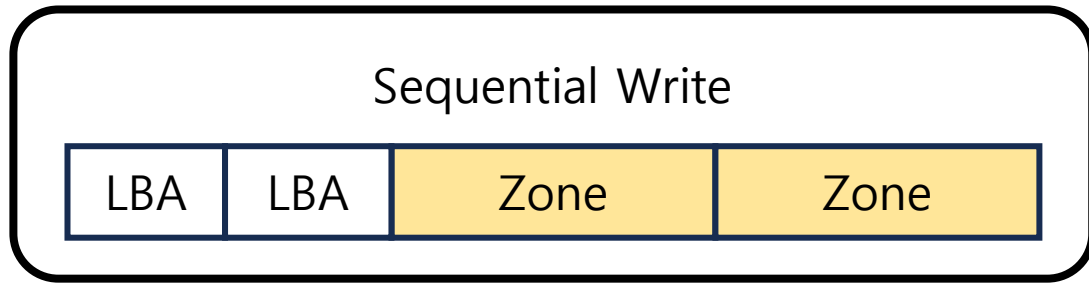
ZNS (Zoned Namespace)

- 특정 미디어 종류에 종속되지 않는 Interface
 - OCSSD: 대부분 작업을 Host에 위임 → SSD 구현 변경 시 SW 수정/관리
 - ZNS: Host는 Block 배치 작업을 부여 + 미디어 고유 특성은 SSD
- SSD의 특성과 장치 Interface간 불일치 제거
 - ZNS: Zoned Storage Model 사용



Problem Background – ZNS

ZNS (NVMe SSD 표준 Interface)



- **Zone의 기본 특성**
 - Sequential Write
 - Overwrite -> Zone Erase
- **기타 특징**
 - Zone의 경계와 실제 물리 미디어 경계 정렬
 - 데이터 관리 → Host / 미디어 관리 → SSD
 - Device-side GC와 Over-Provisioning X

SSD 전체 용량(2TB) -> 4회 overwrite

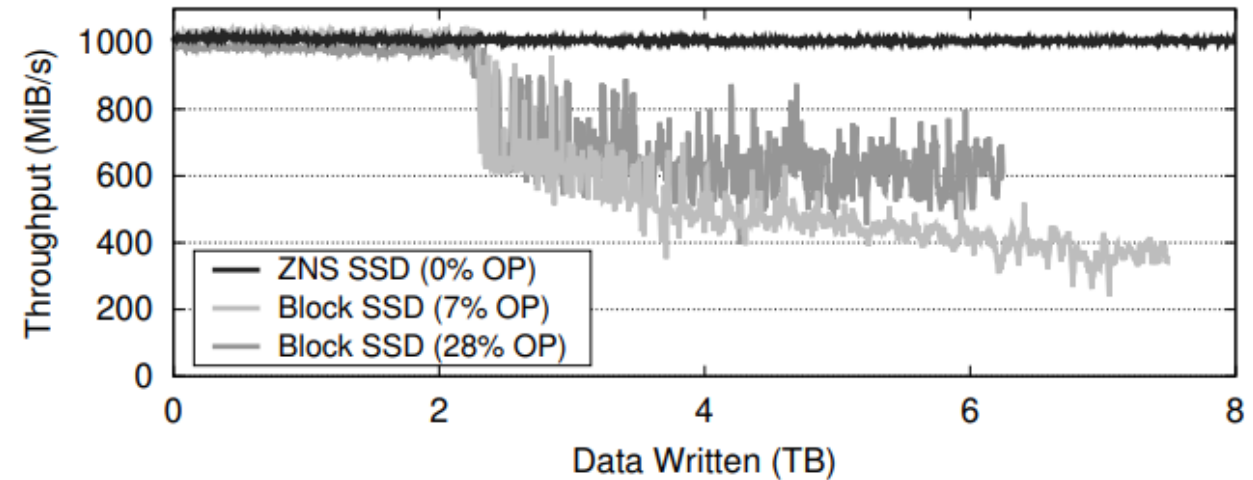
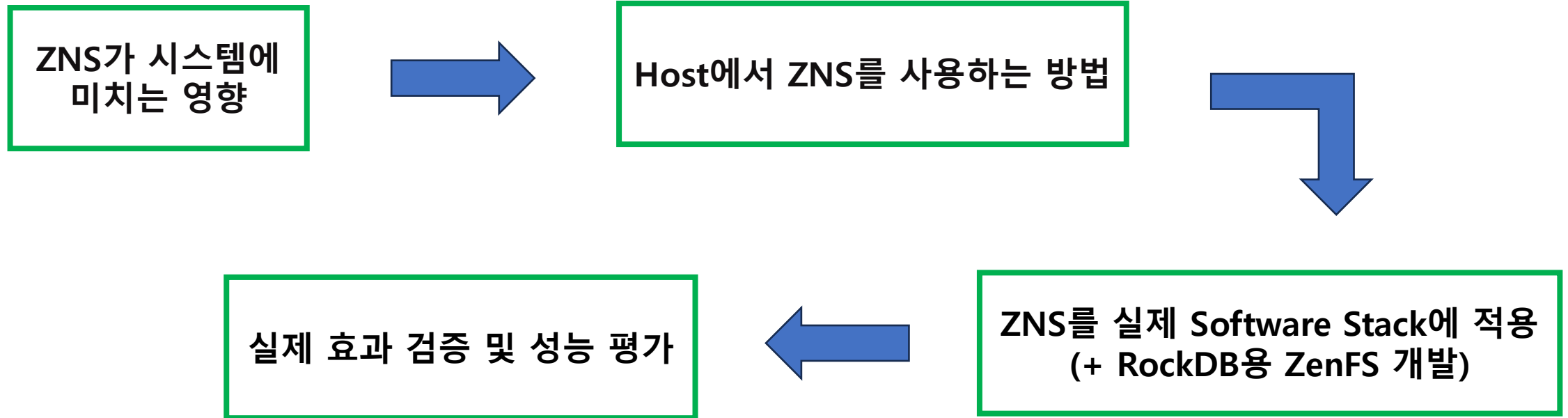


Figure 1: Throughput of a multi-threaded write workload that overwrites usable SSD capacity four times. The SSDs all have 2 TB raw media and share the same hardware platform.

ZNS에는 GC와 OP가 X → 높은 처리량 유지

Paper Roadmap

ZNS의 잠재적 이점을 바탕으로, **실제 시스템에 미치는 영향과 적용 방법을 분석하고 구현을 통해 성능을 검증**

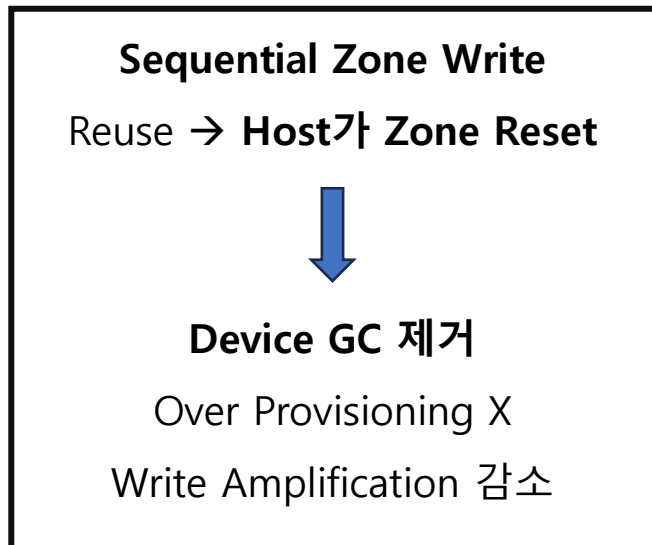


Evolving Towards ZNS

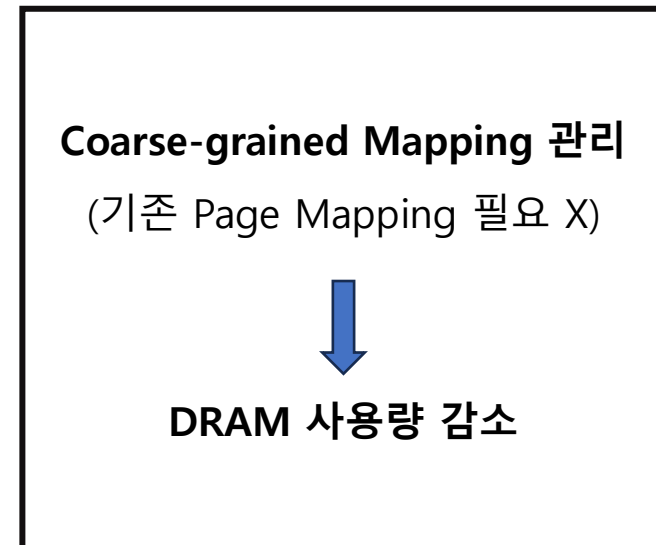
ZNS의 Hardware Impact

ZNS
↓
Impact

1. Simplified Data Management

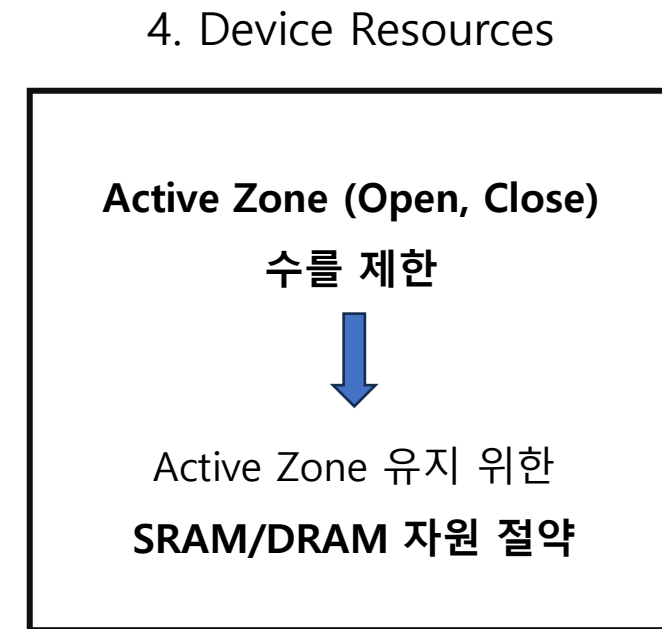
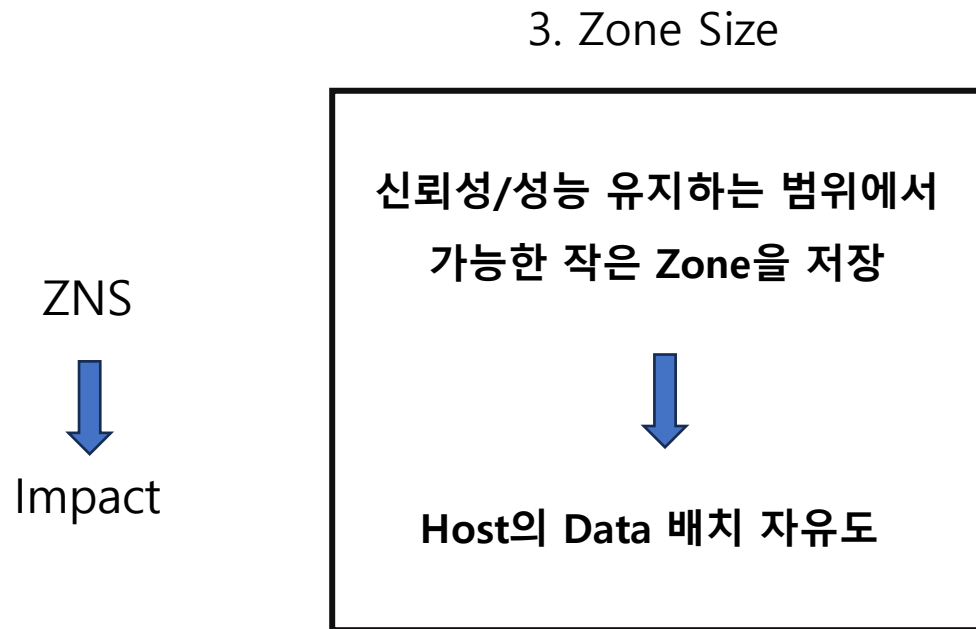


2. Mapping Table



Evolving Towards ZNS

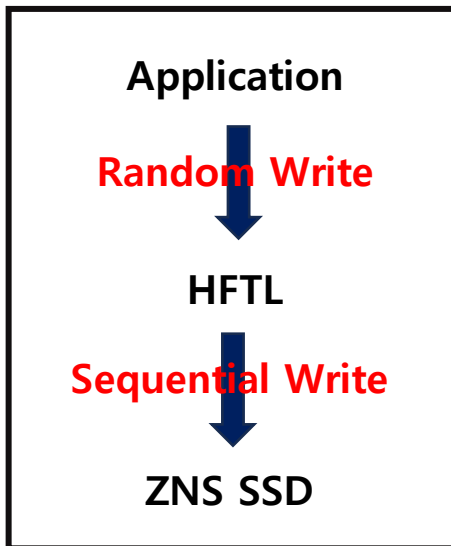
ZNS의 Hardware Impact



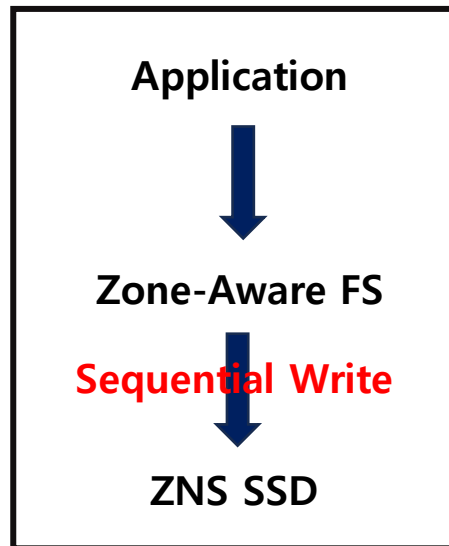
Evolving Towards ZNS

ZNS의 Software Adoption

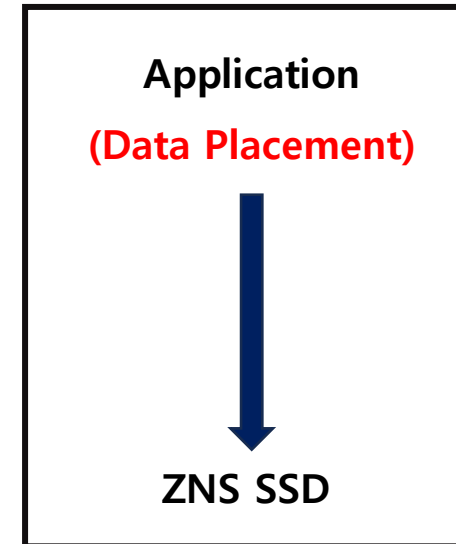
1. HTFL



2. File System



3. End-to-End



Evolving Towards ZNS

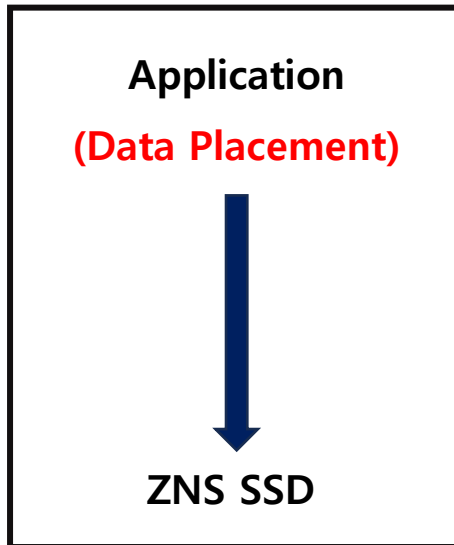
ZNS의 Software Adoption

WHY?

HFTL → Mapping과 GC + Host의 CPU/DRAM

File System → FS가 Data Placement와 GC와 Mapping에 준하는 관리 작업

3. End-to-End



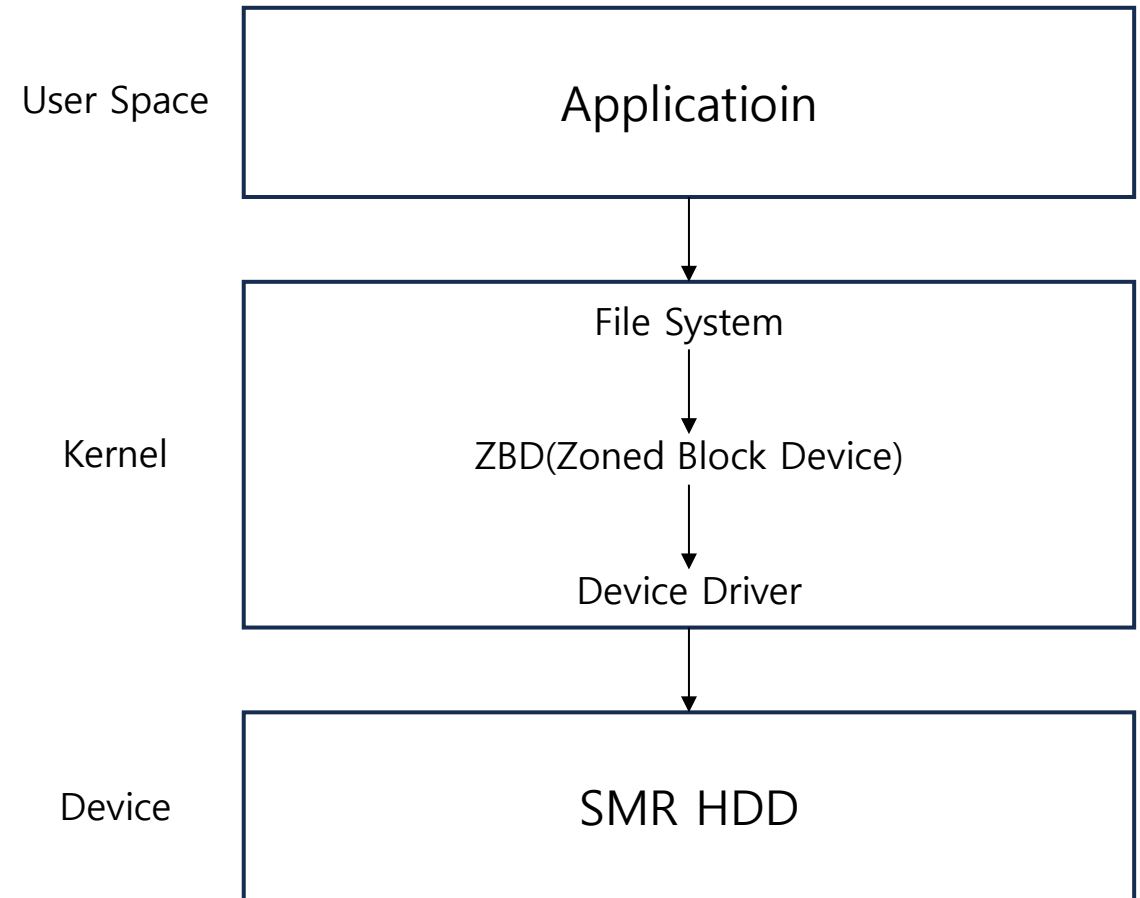
- Application이 Data Placement 직접 관리
- File System, Translation layer의 **indirection / Management overhead 제거**
- Write Amplification / Throughput / Latency 측면에서 가장 큰 개선 가능성

RocksDB의 Sequential / Immutable SST 구조가 ZNS와 잘 맞음

→ 본 논문에서는 End-to-End 방식의 ZenFS를 개발

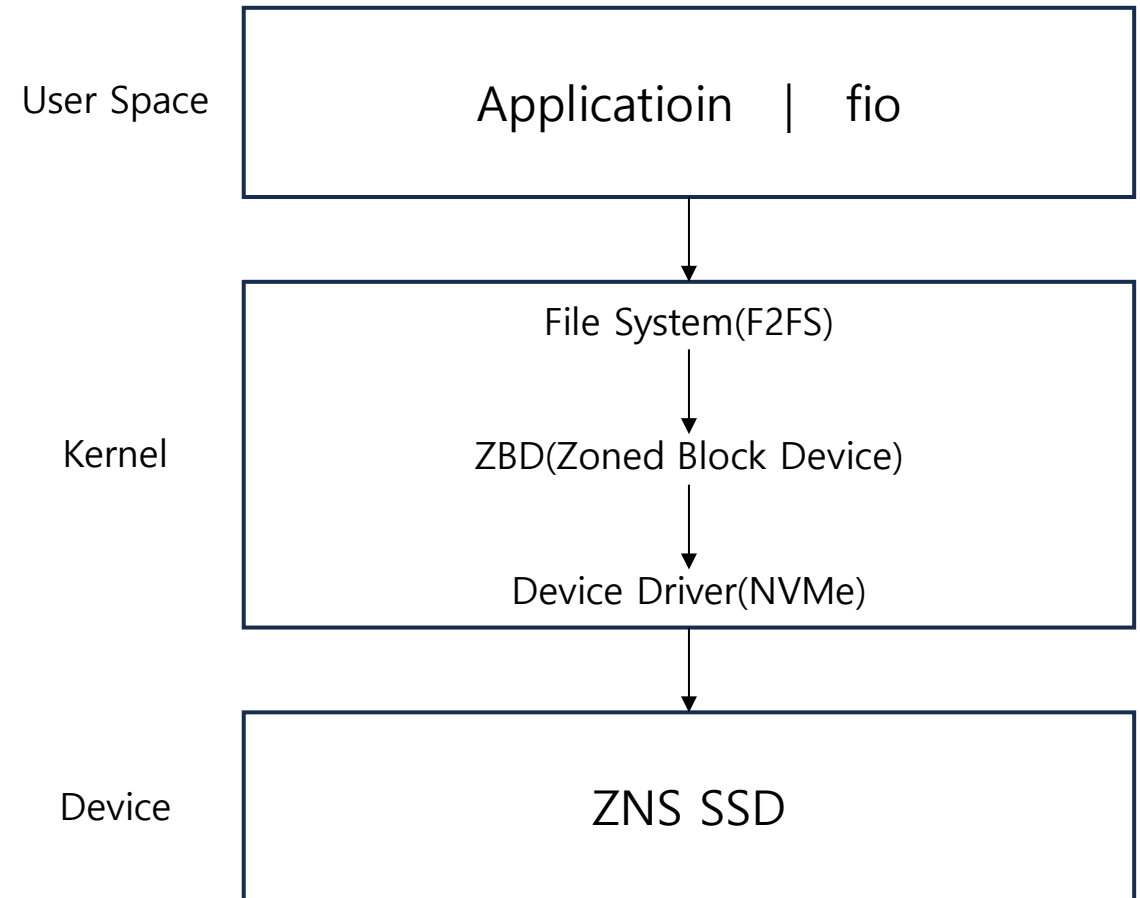
General Linux support

- 기존 SMR HDD의 Storage Stack



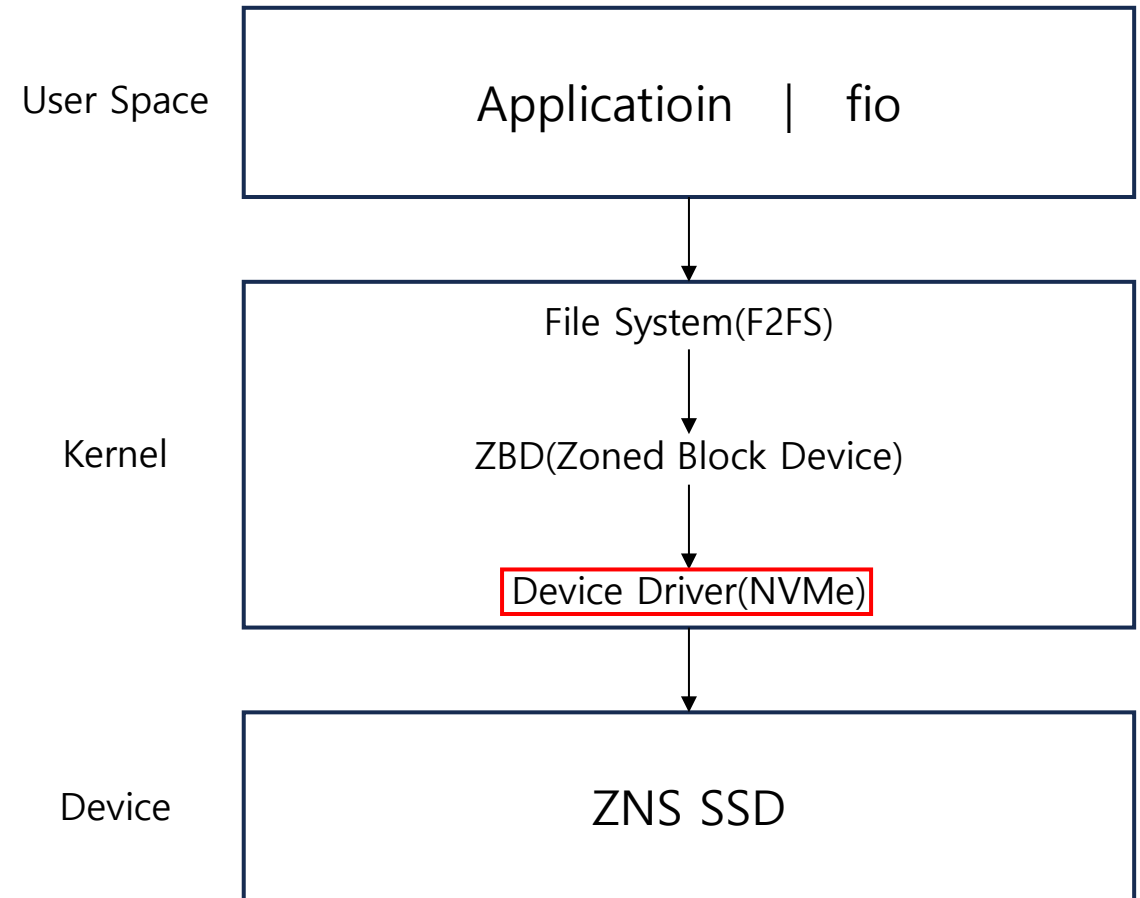
General Linux support

- Zone capacity와 Active Zone Limit을
Linux storage stack에서 표현하고 사용할 수 있도록 확장
- ZNS에서는 Zone Size 전체가 writeable하다는 보장이 없으니,
실제 writeable 범위인 Zone Capacity를 host가 알아야 함
- ZNS SSD는 동시에 Active 상태로 유지할 수 있는
Zone 수가 제한되므로, host software가
이 제한을 인지하고 준수해야 함
- fio(Flexible I/O Tester): 벤치마크 툴



General Linux support – Zone capacity

- ZNS SSD 탐지 및 ZBD 등록

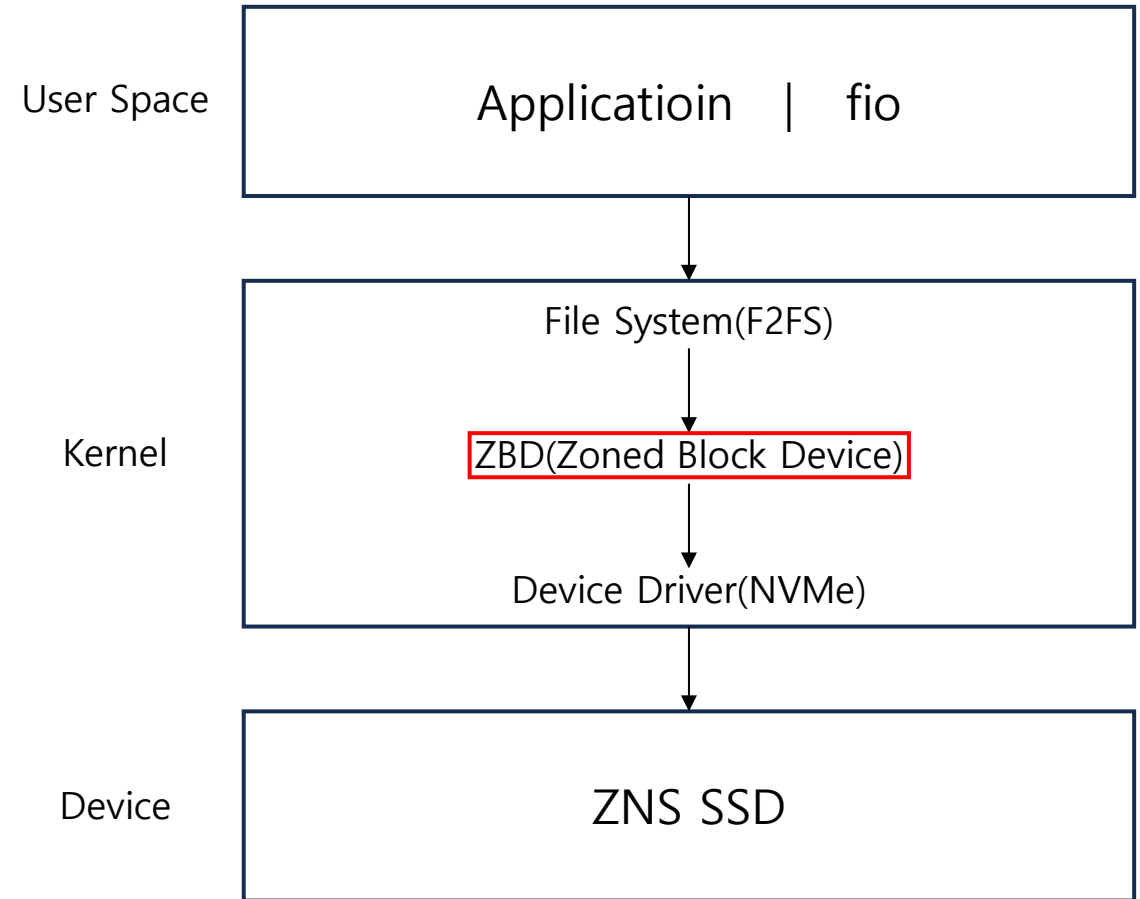


General Linux support – Zone capacity

- ZBD는 각 Zone에 대한 정보를 Zone Descriptor으로 kernel memory에 유지

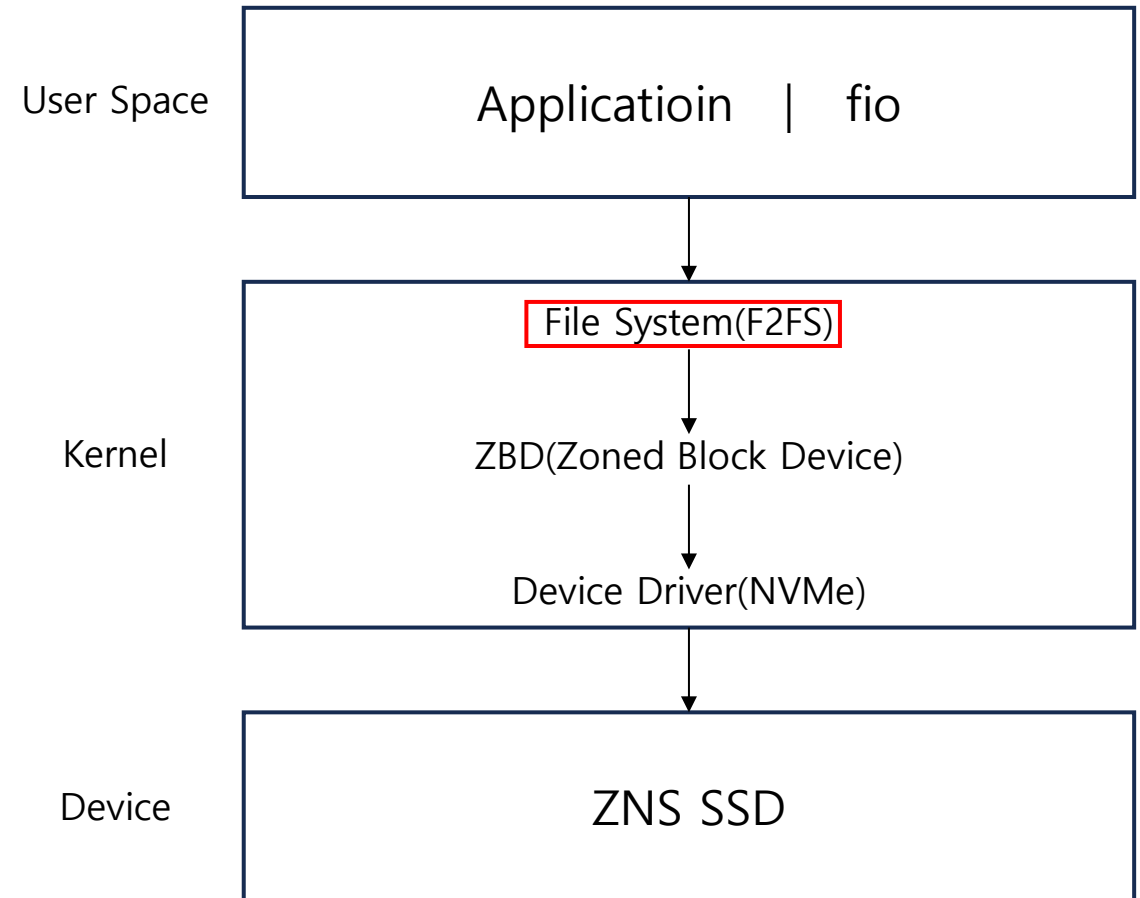
Zone Descriptor 확장

- Zone Capacity 필드 추가
- Versioning 도입

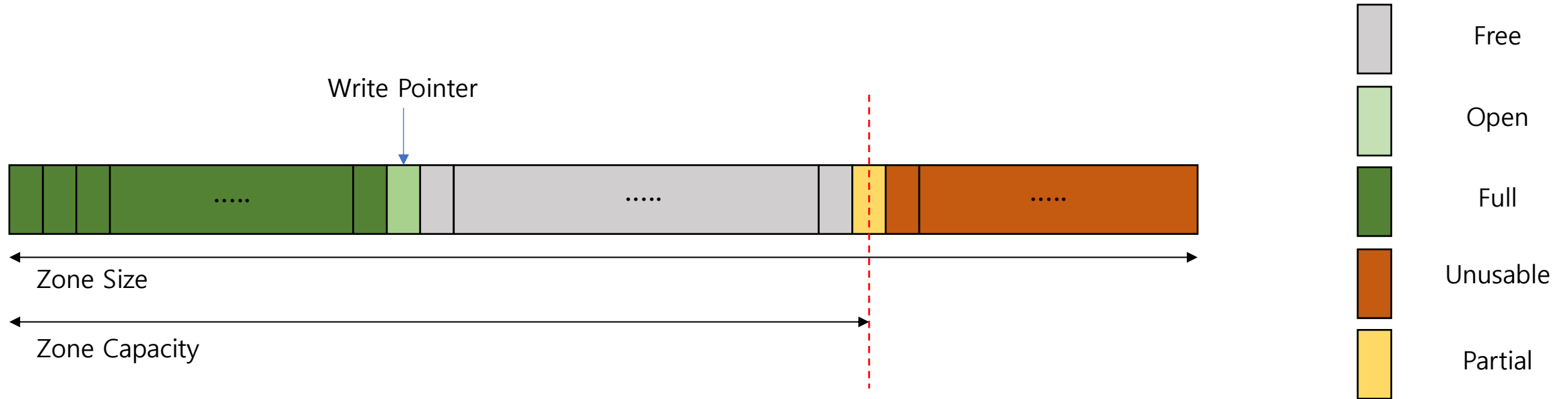


General Linux support – Zone capacity

- F2FS는 segment 단위로 공간을 관리함
- 기존 F2FS segment state
 - > free / open / full
- ZNS Zone Capacity 지원 후
 - > free / open / full / unusable / partial



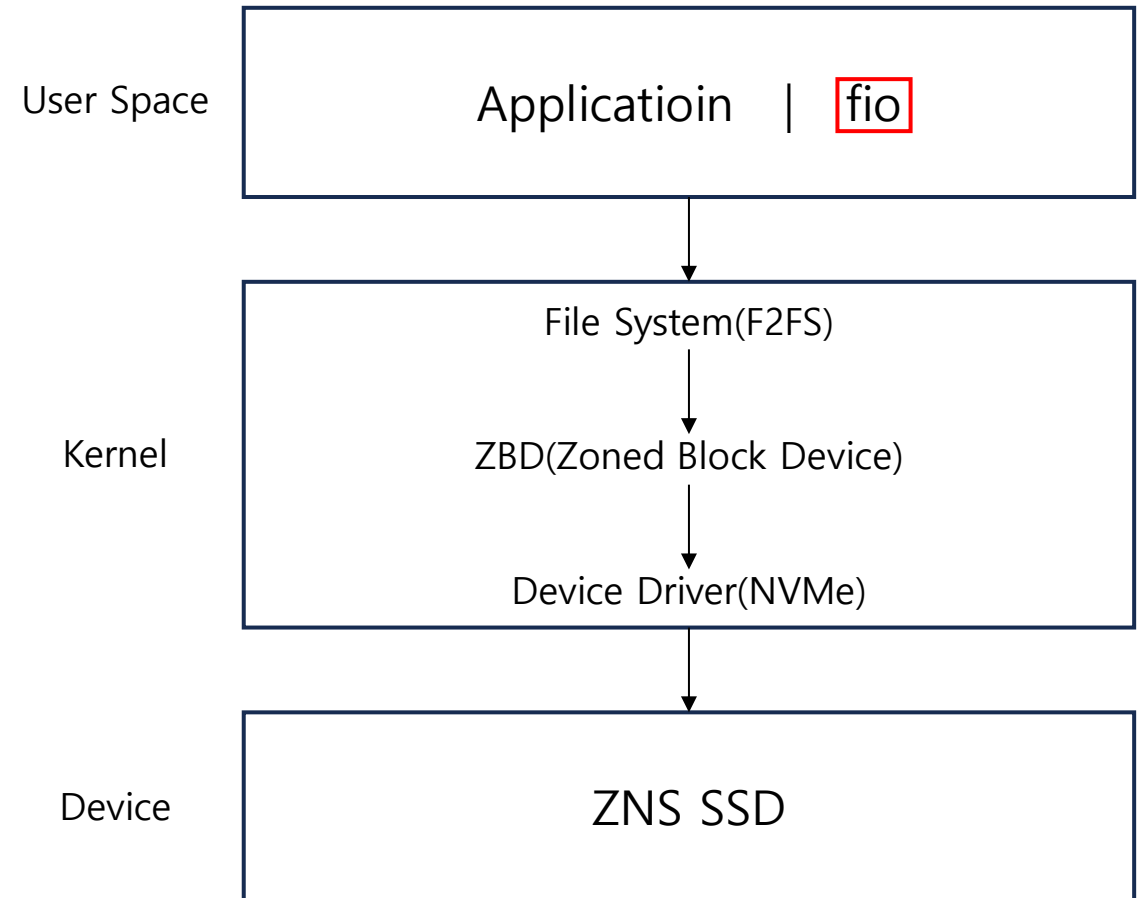
General Linux support – Zone capacity



- Zone Capacity를 넘어서 사용할 수 없는 Segment -> Unusable
- Zone Capacity 경계와 Segment 경계의 misalign으로 인해 일부만 사용 가능한 Segment -> Partial

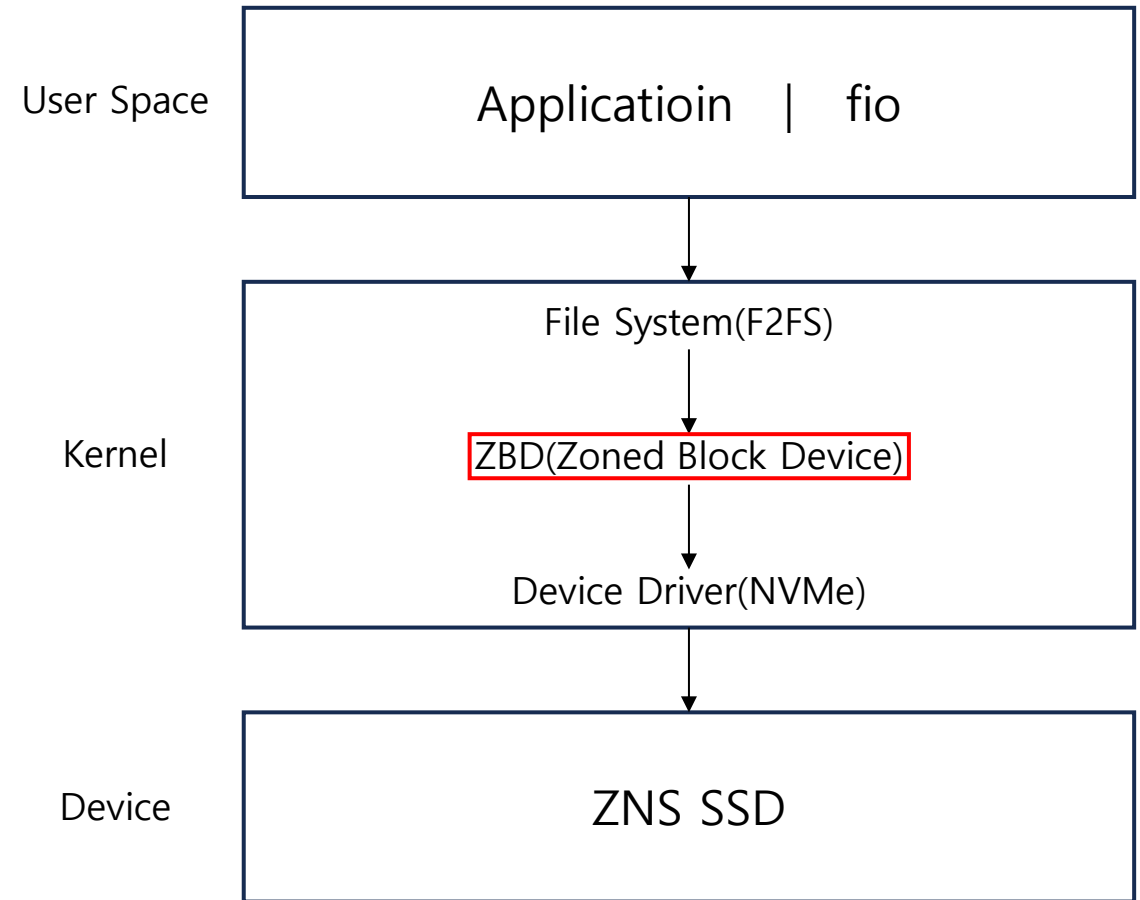
General Linux support – Zone capacity

- 각 Zone의 Zone Capacity 경계를 넘는 write I/O가 발생하지 않도록 수정



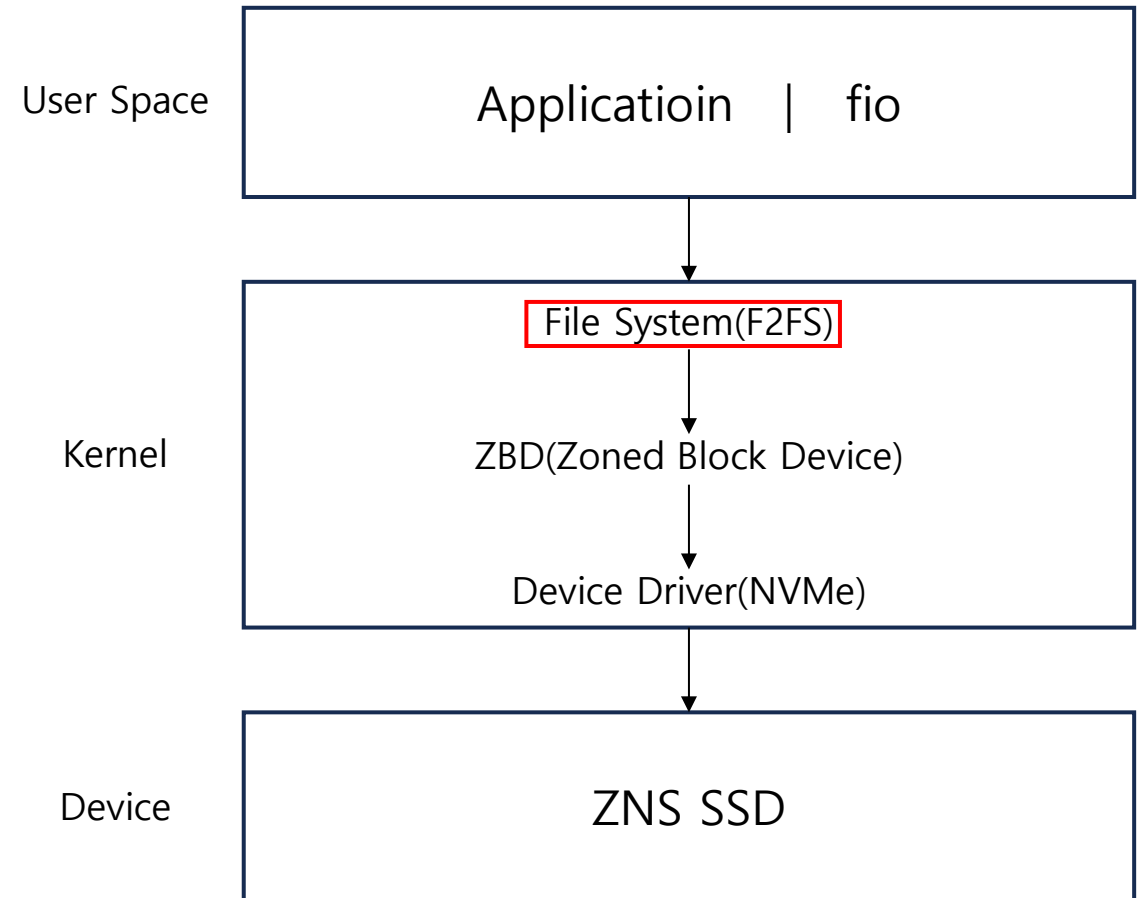
General Linux support – Active Zone Limit

- ZNS SSD 탐지 시 Active Zone Limit을 확인하고, Kernel/User-space API를 통해 노출



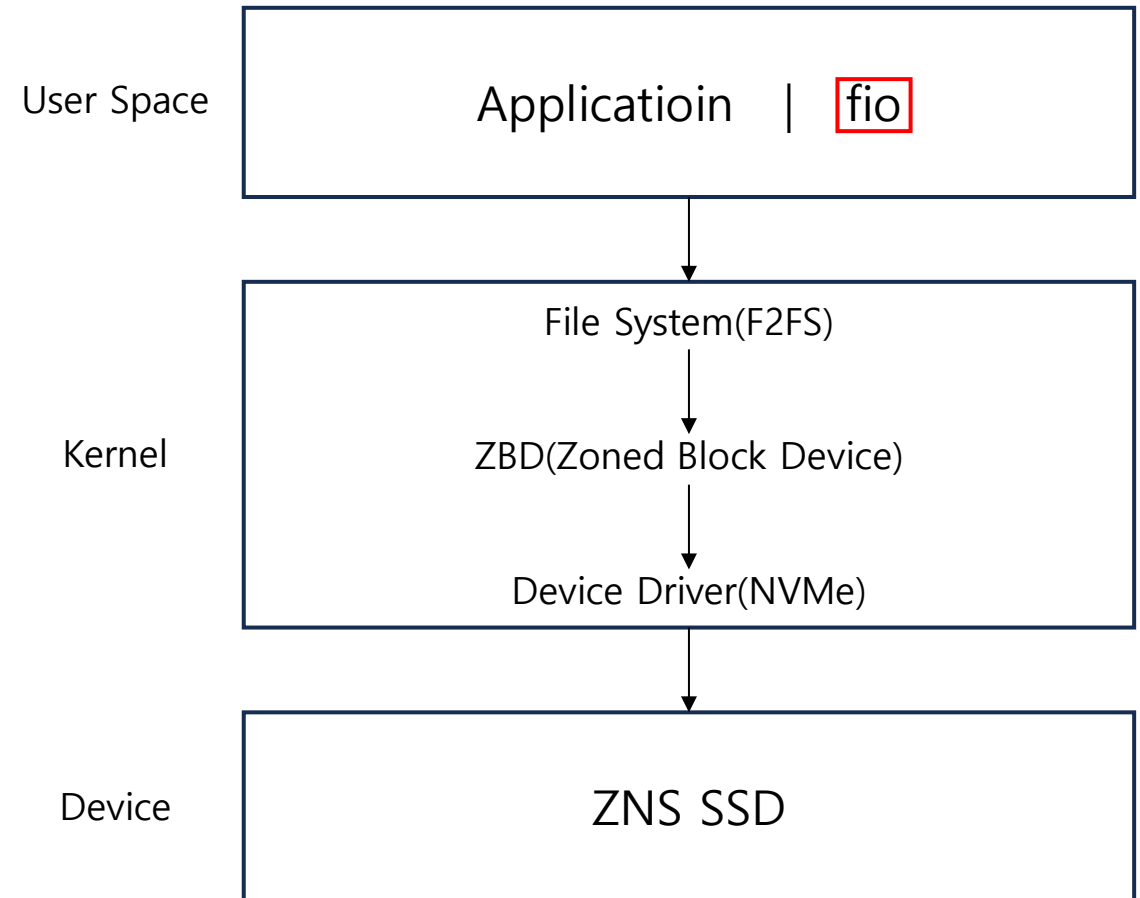
General Linux support – Active Zone Limit

- F2FS는 동시에 OPEN 가능한 Segment를 최대 6개로 제한
- Active Zone Limit이 6보다 작으면,
OPEN Segment 수를 Active Zone Limit에 맞게 축소



General Linux support – Active Zone Limit

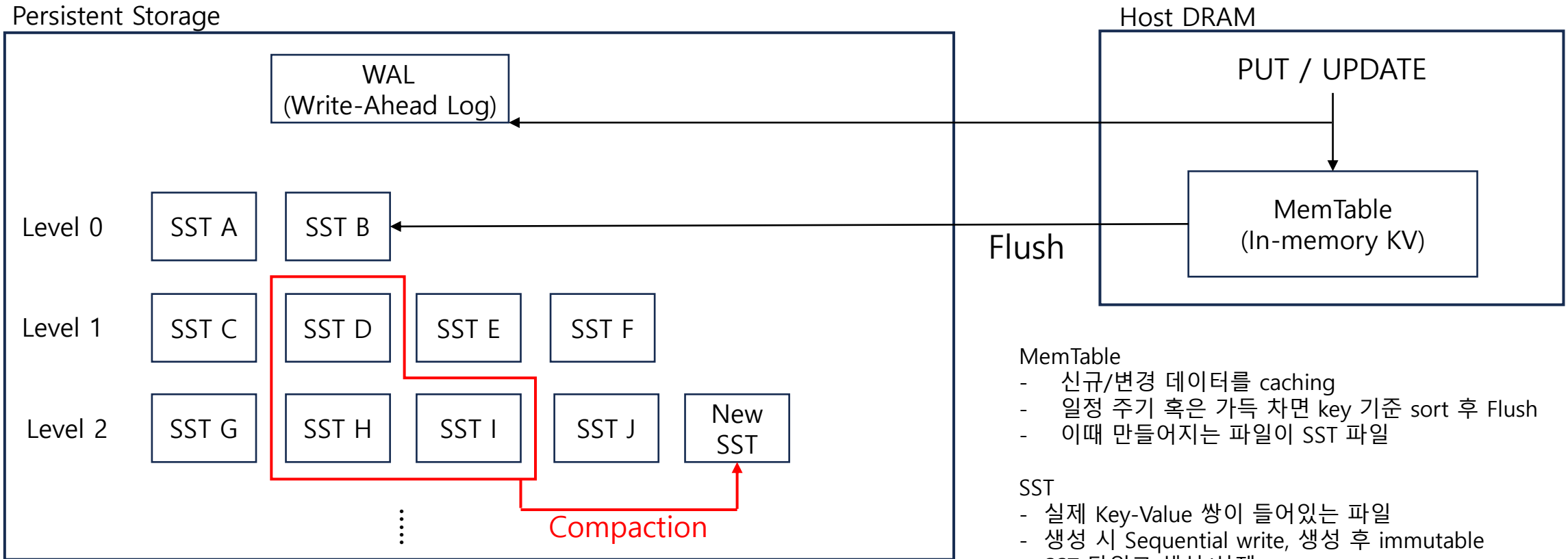
- Active Zone Limit 지원을 위한 fio 수정은 없음
- 사용자가 직접 제한을 지켜야 함



RocksDB – Why RocksDB?

- ZNS
 - Sequential Write
 - In-place overwrite 불가
- RocksDB
 - SST file -> sequential write
 - SST는 생성 후 immutable
 - Update/Compaction시 덮어쓰지 않고, 새로운 SST 생성
 - 이 때도 sequential write

RocksDB – SST File & LSM Tree



MemTable

- 신규/변경 데이터를 caching
- 일정 주기 혹은 가득 차면 key 기준 sort 후 Flush
- 이때 만들어지는 파일이 SST 파일

SST

- 실제 Key-Value 쌍이 들어있는 파일
- 생성 시 Sequential write, 생성 후 immutable
- SST 단위로 생성/삭제

Compaction

- 어떠한 레벨(L_i)의 하나 혹은 그 이상의 SST 파일과, 관련된 다음 레벨(L_{i+1})의 파일들을 merge
- Key 기준 sort하여 새 SST 파일을 만든다(L_{i+1})
- 이 파일은 merge 대상이었던 파일들을 대체함
- 이 또한 Sequential Write

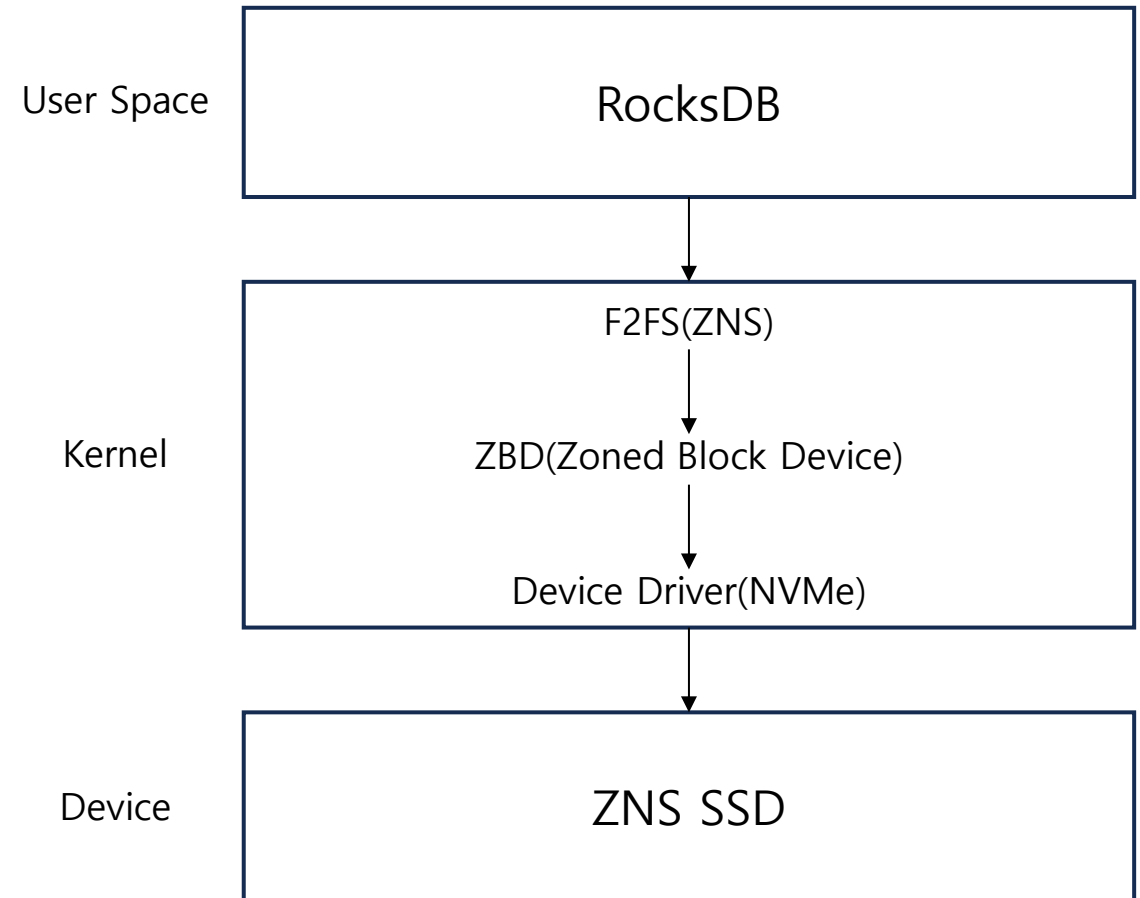
Limitation of Filesystem-based ZNS support

RocksDB

- Write/Update 대상 파일에 대한 특성을 알고있음
 - SST / WAL
 - SST Level
 - Write Lifetime Hint

F2FS(ZNS)

- ZNS-aware Filesystem
- 실제 extent / free-space / zone placement 관리
- ZNS constraint를 준수하여 ZNS SSD 사용 가능
- RocksDB가 Zone Placement를 직접 제어할 수 없음
 - > End-to-End Data Placement 불가

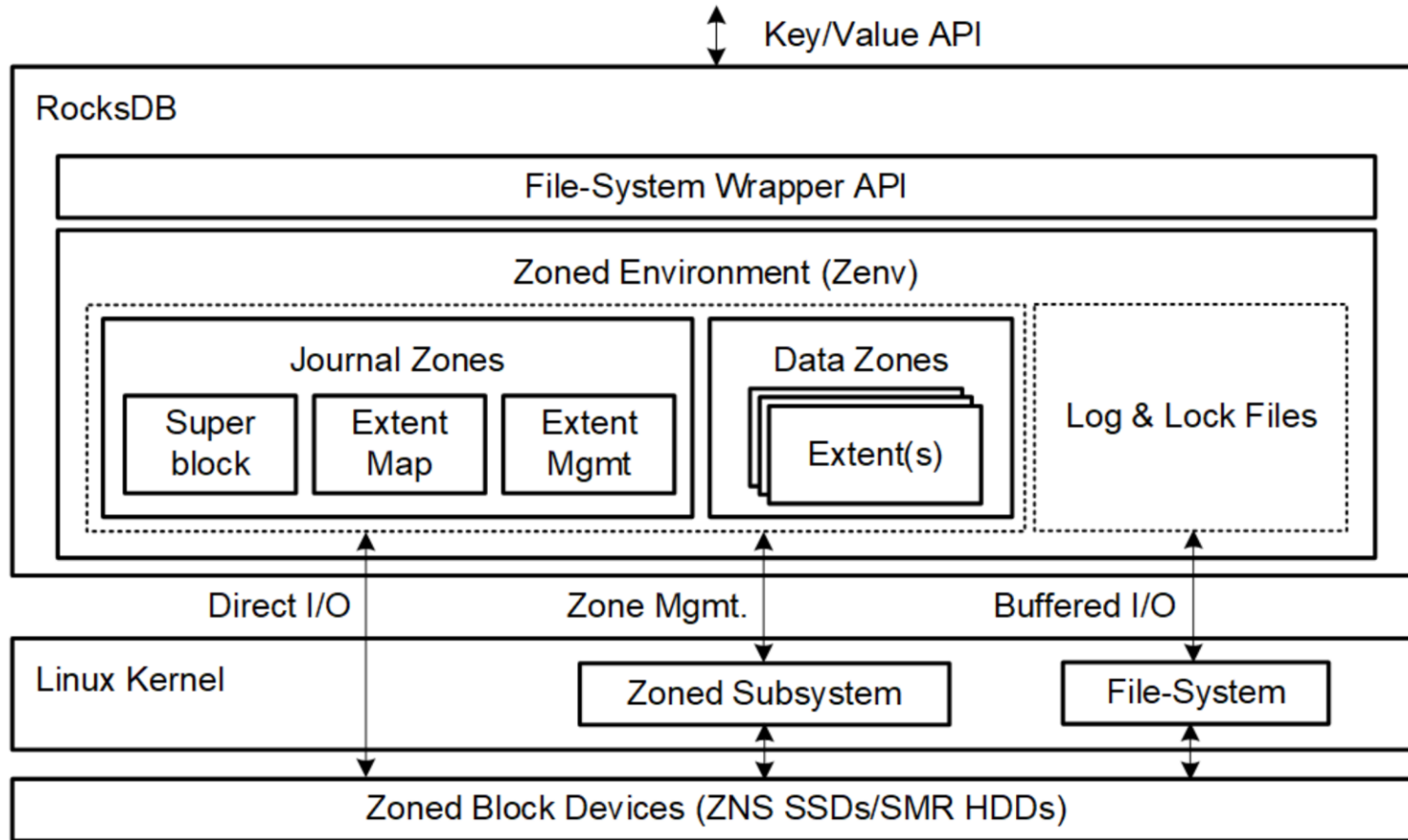


ZenFS – Overview

ZenFS

- RocksDB를 위한 Zone-aware Storage Backend
- RocksDB의 File System Wrapper API에 Storage Backend로 연결
- 기존 파일시스템 대신 minimal on-disk file system 기능을 직접 수행
- ZNS의 sequential-write constraint를 준수하면서 파일 데이터를 Zone에 직접 배치
-> End-to-End Data Placement 가능

ZenFS - Architecture



Journal Zone

- File - Zone 매핑 등 메타데이터 저장 및 복구

Data Zone

- RocksDB 파일 데이터를 Extent 단위로 배치·저장

Direct Zone Management

- ZenFS가 Extent와 Zone 배치를 직접 관리
-> End-to-End Data Placement

Extent

- Variable-sized
- Block-aligned
- Contiguous region
- Sequentially written

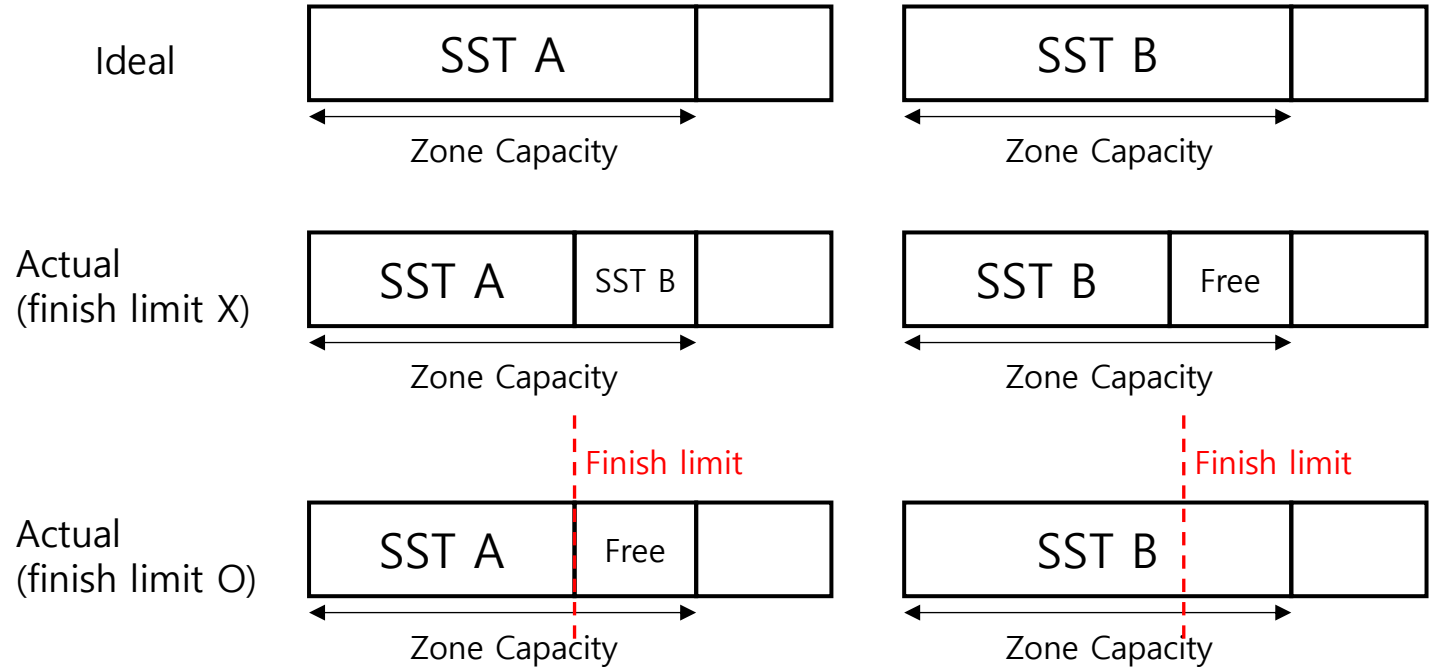
Figure 4: ZenFS Architecture

Data Zones – Writeable Capacity

[Problem]

SST File Size

- Recommendation only
- Compression & Compaction
- Exact alignment difficult



Finish Limit

- Remaining capacity가 threshold 이하이면 Zone Finish
- File-size variation을 허용하면서 File Separation by Zone 유지 가능

Data Zones – Writeable Capacity

Larger SST Files?

- 보통 Zone Capacity가 SST 파일 크기보다 크니 Zone capacity에 더 가깝게 SST를 배치하기 위해 SST 크기를 키워도 RocksDB에 악영향을 끼치지 않는지 실험함

Fillrandom

- Random key-value write
- write-heavy benchmark
- Runtime, Write Amp. 측정

Readwhilewriting

- 읽기와 동시에 쓰기를 하며 read tail latency 측정
- RW: Write rate 제한 없음
- RWL: Write를 20MiB/s로 제한

SST file size	128 MiB	256 MiB	512 MiB
Write Amp.	11.9×	12.0×	12.0×
Runtime (s)	15,430	15,461	14,918
99.99 RW lat (ms)	102	102	105
99.99 RWL lat (ms)	77	73	73

Table 2: RocksDB's write amplification and runtime during the *fillrandom* benchmark, and 99.99-th percentile tail latencies during the *readwhilewriting* benchmark with no rate limits (RW) and with writes rate-limited to 20 MiB/s (RWL), with different SST file sizes.

-> SST size를 128 -> 512MiB로 증가시켜도 유의미한 성능 저하 없음

Data Zones – Data Zone Selection

[Goal]

서로 다른 lifetime을 가진 파일의 extent가
같은 Zone에 배치되면 일부 파일이 삭제되어도
long-lifetime 파일의 Extent가 남아 Zone Reset이 지연

-> 비슷한 lifetime의 파일 extent를 같은 Zone에 배치해야 함

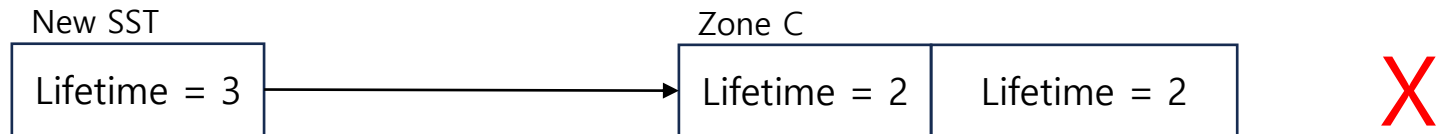
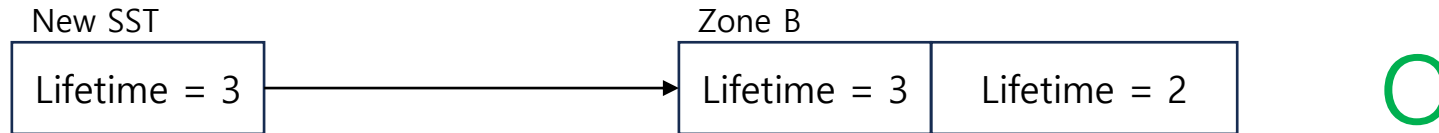
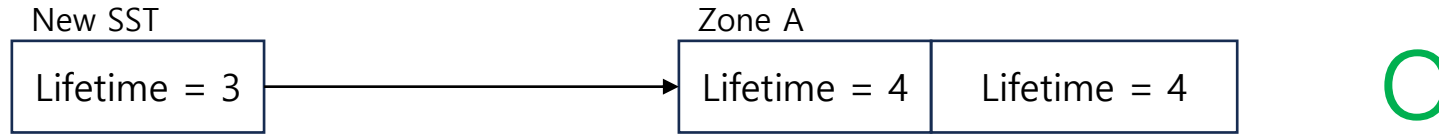
Zone Selection Algorithm

- best-effort
- Extent를 기존 Zone의 수명을 늘리지 않는 선에서 비슷한 lifetime의 Zone에 배치함

Write Lifetime hint

- RocksDB가 파일 write 전에 lifetime hint 지정
- ZenFS가 Zone 선택에 활용(ZenFS 아니여도 가능)

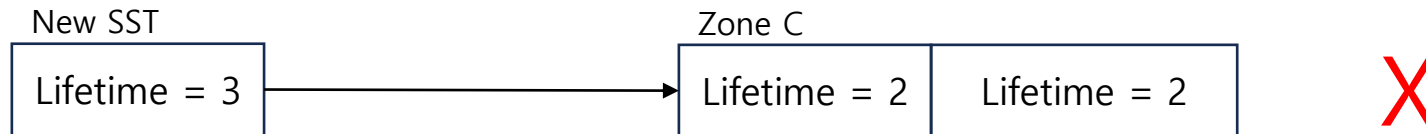
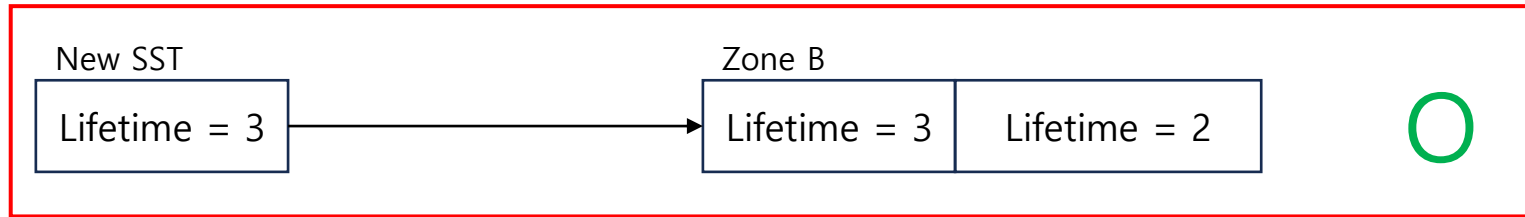
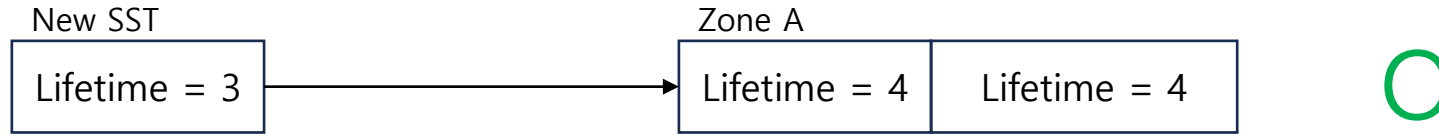
Zone Selection Algorithm



1. Write할 extent의 lifetime이, 저장하려는 zone의 max lifetime보다 길면 안 됨
(Zone의 lifetime을 늘리지 않기 위함)

- Lifetime에 숫자를 넣은 것은 오직 예시를 위함

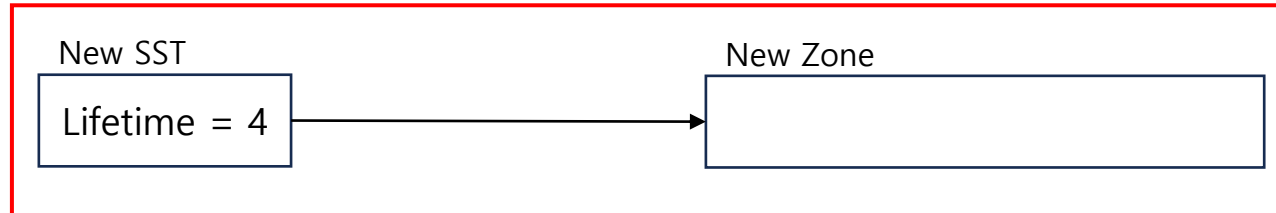
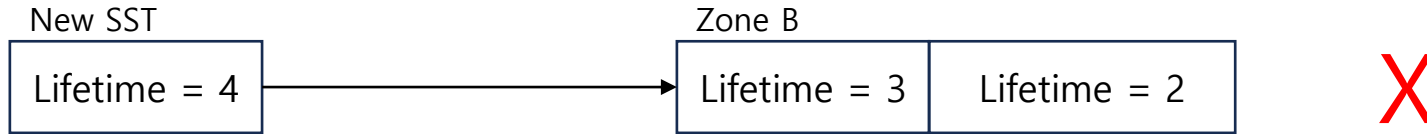
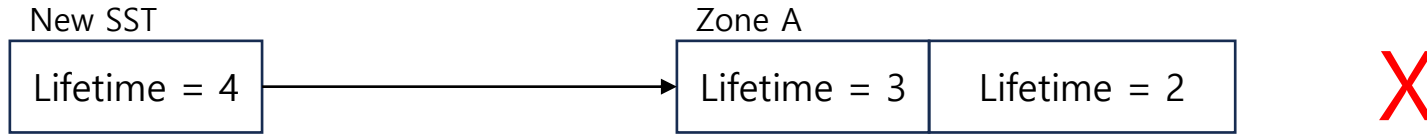
Zone Selection Algorithm



2. 만약 이 조건을 만족하는 zone이 여러 개 있다면, closest match가 사용됨

- Lifetime에 숫자를 넣은 것은 오직 예시를 위함

Zone Selection Algorithm



3. 조건을 만족하는 zone이 없다면, empty zone을 할당함
파일이 할당된 zone의 remaining capacity를 전부 채우면,
같은 알고리즘으로 다른 zone을 할당함

- Lifetime에 숫자를 넣은 것은 오직 예시를 위함

Active Zone Limits

- ZenFS는 ZNS SSD의 Active Zone Limit을 준수해야 함
- 최소 3개의 Active Zone 필요
 - Journal / WAL / Compaction
- Concurrent Compaction 수를 조절하여 Active Zone 사용량 제어
- 실험 결과
 - 6 Active Zones: RocksDB 동작 가능, 단 Write Performance 제한됨
 - 12개 초과: 유의미한 추가 성능 향상 없음

Direct I/O & Buffered Writes

- SST file
 - Sequential write, immutable
 - Kernel Page cache를 우회하여 direct I/O write를 진행함
- WAL
 - ZenFS 내부 memory Buffer 사용
 - Full / Close / Flush request시 Device에 기록

Evaluation - Setup

- ZNS SSD와 Block interface 둘 다로 expose하고 사용 가능한 SSD 사용(즉, 하드웨어 자체는 같음)
 - 그 종류는 다음과 같음

SSD Interface	Conv.	Conv.	ZNS
Media Capacity	2 TiB	2 TiB	2 TiB
Host Capacity	1.92 TB	1.60 TB	2 TB
Over-provisioning	7%	28%	0%
Placement Type	None	None	Zones
Max Active Zones	N/A	N/A	14
Zone Size	N/A	N/A	2048 MiB
Zone Capacity	N/A	N/A	1077 MiB

Table 3: Feature summary of the evaluated SSDs

Evaluation - Setup

- 기타 사양
 - Dell R7515
 - AMD EPYC 7302P, 16 cores
 - DRAM 128 GB
 - Ubuntu 20.04
 - Linux kernel 5.9
 - RocksDB 6.12 + ZenFS backend

Raw I/O Characteristics

raw device 수준에서 ZNS interface의 효과를 확인하기 위한 실험

Sustained Write Throughput[Figure 5(a)]

- SSD GC로 인한 throughput 영향을 측정
- Write rate를 0에서 1GiB/s까지 올리면서 SSD가 그 목표 속도를 실제로 따라가는지 측정 (64KiB Sequential write)

Random Reads and Writes[Figure 5(b)]

- 앞선 Write workload에 SSD 전체 주소 공간 대상 random 4KiB read를 더함
- Average random-read latency 측정

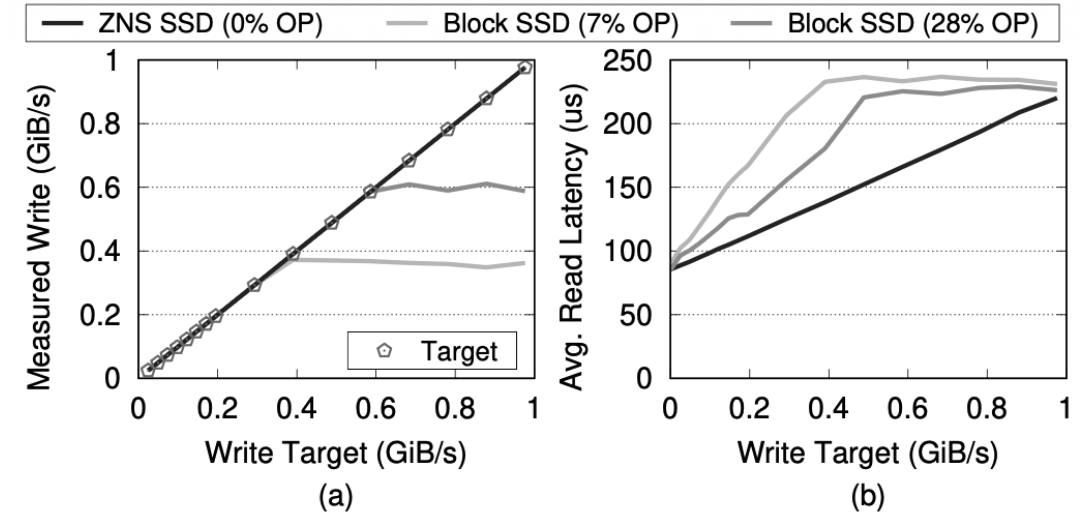


Figure 5: Measured (a) write throughput and (b) average random read latency during rate-limited writes.

Device-side GC 제거 -> 높은 Write Throughput 및 낮은 Average Random Read Latency

RocksDB - Setup

Goal:

1. RocksDB workload상에서 ZNS의 장점 확인
2. ZenFS와 F2FS(ZNS)를 비롯한 여러 storage stack과의 비교

Storage Stack

- BlockSSD
 - RocksDB + XFS + Block SSD
 - RocksDB + F2FS + Block SSD
- ZNS SSD
 - RocksDB + F2FS(ZNS) + ZNS SSD
 - RocksDB + ZenFS + ZNS SSD

WorkLoad

- Fillrandom
 - Write-intensive
 - SSD를 여러 번 채우면서 후속 실험을 위한 pre-conditioning
- Overwrite
 - 기존 value를 random overwrite
- Randomread
- Readwhilewriting
- Readwhilewriting + rate limit
 - Write rate 제한

공통 조건

- Benchmark dataset 규모: SSD의 80%
- 대부분 결과는 Block SSD 28% OP 기준임
Benchmark가 사용하지 않는 20%까지 더하면 사실상 약 48% OP
- 일반 setup의 SST target size: 128 MiB
- ZenFS:
SST target size를 zone capacity에 맞게 설정

RocksDB – Write Throughput

4가지 setup에 대하여 fillrandom과 overwrite throughput 비교

- 전반적으로 XFS/F2FS < F2FS(ZNS) < ZenFS
- GC 영향이 더 큰 overwrite에서 차이가 두드러짐

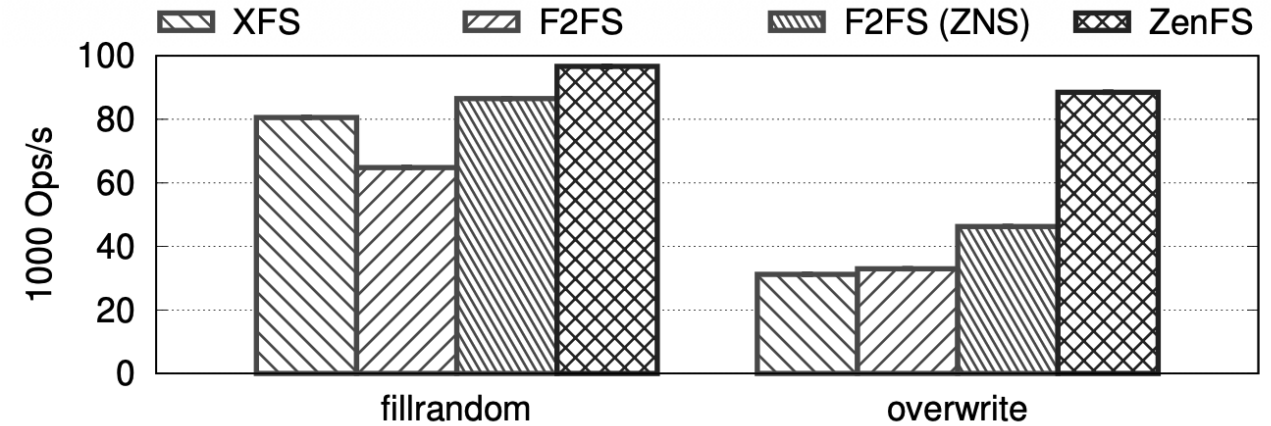


Figure 6: Throughput of RocksDB with write-heavy benchmarks—*fillrandom* followed by *overwrite* using the block-interface SSD with 28% OP and the ZNS SSD.

	XFS	F2FS	F2FS(ZNS)	ZenFS
Device-side Write amp.	2.0	2.4	1.0	1.0

RocksDB – Read While Writing

다음 workload에 대해 throughput과 tail latency 측정

- (a): SSD precondition 후 random read
- (b): (a) + overwrite(write rate = 20MiB/S)
- (C): (b)에서 write rate limit 제거

- Block SSD with 28% OP

결과

- (a)
 - Write가 없어 throughput과 p99 latency에서는 큰 차이가 없음
 - 이후 실험과 비교하기 위한 baseline 성격
- (b)
 - ZNS SSD와 Block SSD간의 p99.99 tail latency 차이가 크게 나타남
 - > device GC 여부로 인한 tail latency 차이
- (c)
 - Write throughput에서 ZenFS가 XFS/F2FS보다 약 2배 높은 Throuput을 보임
 - P99.99 latency 차이가 더 커짐

Read/Write 동시 수행 시 높은 Read Throughput과 낮은 Tail Latency

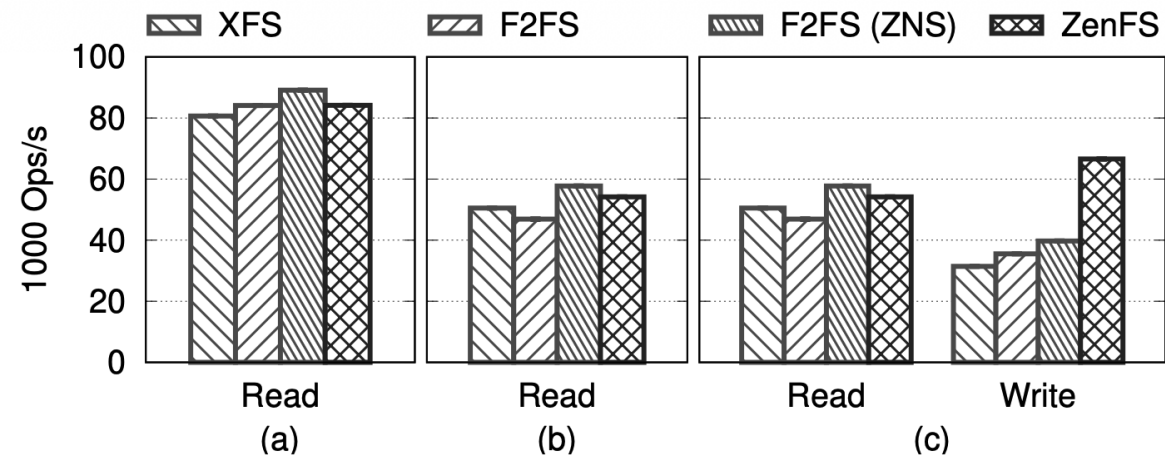


Figure 7: Read/Write throughput of RocksDB

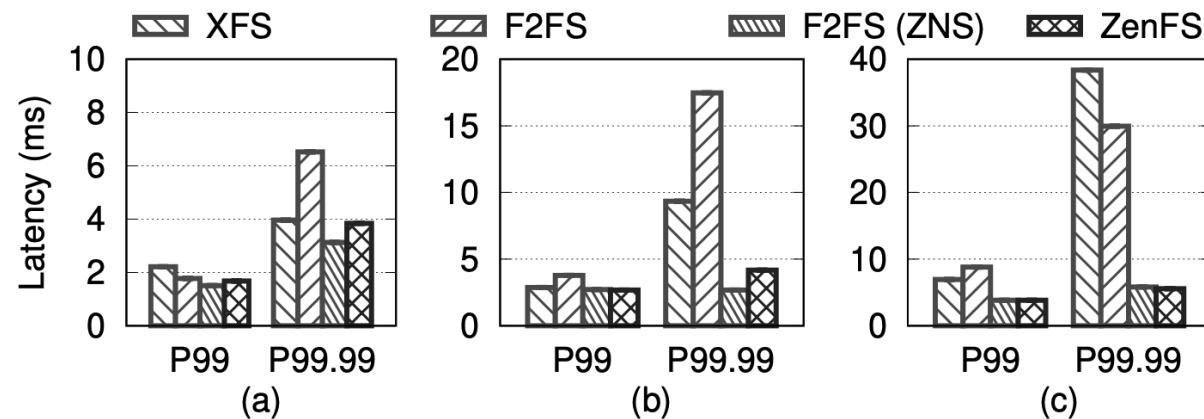


Figure 8: Read tail latency of RocksDB

RocksDB – Impact of Over-Provisioning

실험 목적:

- Block SSD의 OP 증가가 RocksDB 성능에 미치는 영향 확인
- OP 증가시 GC 효율 향상 -> Write amp. 감소

XFS/F2FS에 대하여 각각 7%OP와 28% OP 모두 실험

Fillrandom, Overwrite, Rate-limited readwhilewriting 워크로드에서
Throughput 측정

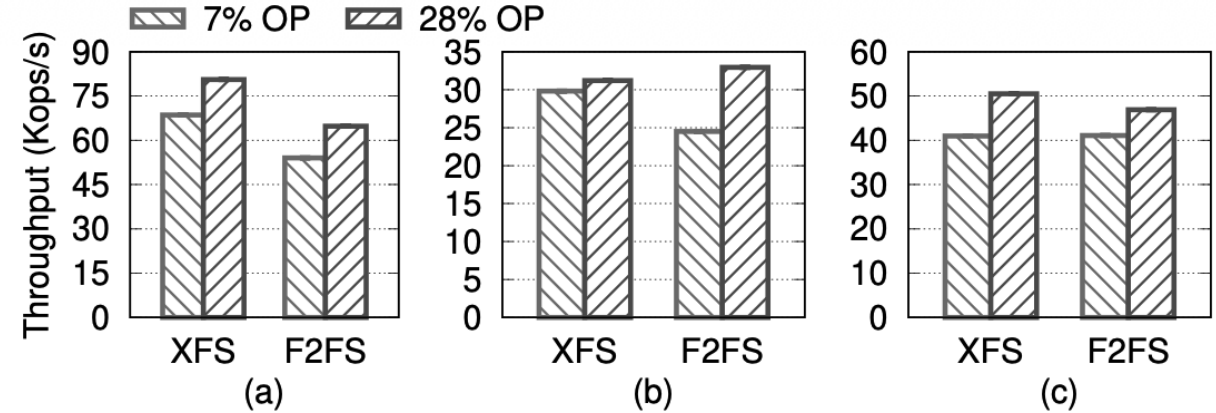


Figure 9: Throughput of RocksDB writes during (a) *fillrandom*, (b) *overwrite*, and (c) *readwhilewriting* benchmark, using the block-interface SSD with 7% OP, as well as 28% OP.

->Block SSD는 OP 증가로 GC 성능을 개선할 수 있지만, 그만큼 Usable Capacity를 희생

RocksDB - Streams

Stream으로 최적화한 Block SSD와 ZNS를 비교하기 위한 실험

Block SSD

- 7% OP
- Streams 지원

각 setup들의 Throughput 비교

- F2FS의 p99 latency가 streams 적용으로 9,435 μ s -> 6,529 μ s으로 개선
- ZenFS는 같은 벤치마크에서 3,734 μ s

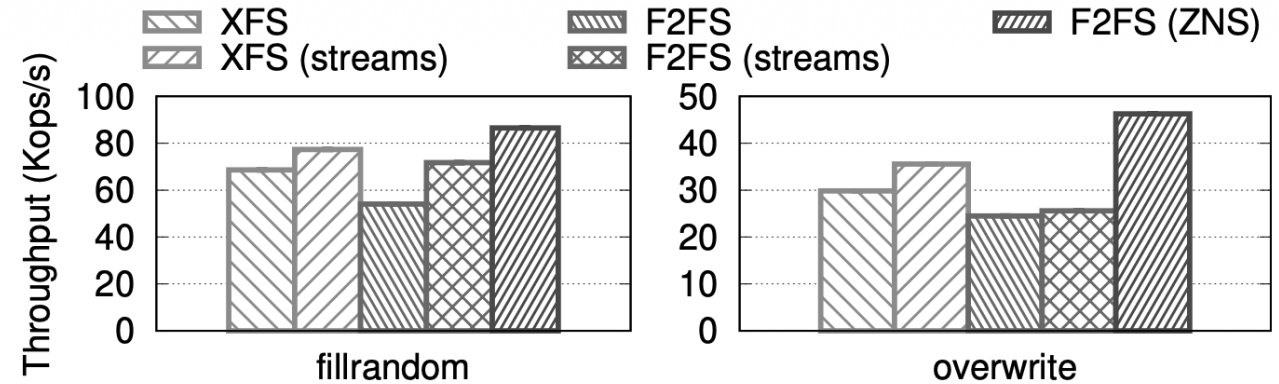


Figure 10: The throughput of RocksDB during the *fillrandom* and *overwrite* benchmarks when executing on an block-interface SSD with 7% OP and stream support.

	Fillrandom	Overwrite
XFS	11%	16%
F2FS	24%	4%

Table: Streams 적용시 Throughput 증가율

-> Streams로 GC를 완화해도, Device-side GC를 제거한 ZNS가 더 높은 Throughput과 낮은 Tail Latency 달성

Conclusion

Related Works

1. Host-Device Collaboration

- 기존 연구 → Host에 더 많은 Flash 제어권 제공
- **ZNS → Data Placement = Host / Media Reliability = SSD**

2. Research Platforms

- 기존 SSD 플랫폼을 기반으로 SSD 특성을 활용 → Block Interface를 개선하는 연구가 진행
- **ZNS는 이러한 연구들을 기반으로 발전한 표준 Interface**

3. Key-Value Store Designs

- 기존 KV Store → 특정 OCSSD, SMR HDD에 종속
- **ZenFS → 일반적인 Zoned Interface 대상 + ZNS SSD, SMR HDD 모두 지원**

Conclusion

Related Works

1. Host-Device Collaboration

- 기존 연구 → Host에 더 많은 Flash 제어권 제공
- ZNS → Data Placement = Host / Media Reliability = SSD

2. Research Platforms

- 기존 SSD 플랫폼을 기반으로 SSD 특성을 활용 → Block Interface를 개선하는 연구가 진행

ZNS는 Data Placement 책임을 Host로 이동시켜 In-device Mapping, GC, OP를 제거하고 성능과 비용 효율을 향상

ZNS에 특화하여 RocksDB를 수정해 ZenFS 구현

→ Write Throughput 향상, Read Tail Latency 감소, Write Amplification 감소

Thank you